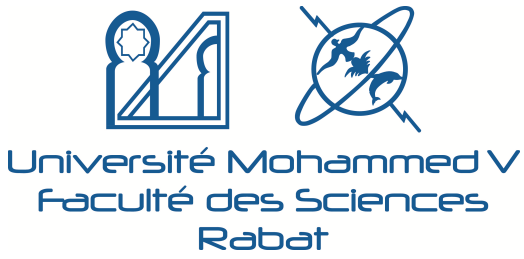
UNIVERSITÉ MOHAMMED V DE RABAT
Faculté des Sciences



Département d'Informatique

Filière Licence Fondamentale
en Sciences Mathématiques et Informatique

PROJET DE FIN D'ÉTUDES

Intitulé :

Système de détection d'intrusions avec Machine Learning

Présenté par :

HANABAL Salma
EL KHABBAOUI Iliass

Soutenu le 20 Juin 2025 devant le Jury

Pr. LOTFI Dounia	Faculté des Sciences - Rabat	*Présidente*
Pr. DRISSI EL MALIANI Ahmed	Faculté des Sciences - Rabat	*Examinateur*
Pr. LAGHMATI Sara	Faculté des Sciences - Rabat	*Encadrante*

Année Universitaire 2024-2025

Remerciements

Nous souhaiterions adresser nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Nos plus vifs remerciements vont à notre encadrante, pour sa disponibilité, sa patience, et la qualité de son accompagnement tout au long du projet. Ses orientations, ses conseils méthodologiques et ses remarques pertinentes ont grandement enrichi notre réflexion et ont guidé l'aboutissement de ce travail.

Nous tenons également à remercier l'ensemble du corps pédagogique pour la qualité de l'enseignement dispensé tout au long de notre parcours. Les connaissances acquises durant notre formation nous ont permis d'aborder ce projet avec les outils et la rigueur nécessaires.

Nous exprimons enfin notre reconnaissance à nos familles et nos amis pour leur soutien constant, leur encouragement moral et leur bienveillance, qui nous ont accompagnés dans les moments de doute comme dans les réussites.

Ce projet a été pour nous bien plus qu'un simple exercice académique : il a constitué une expérience enrichissante, mêlant apprentissage autonome, esprit critique, et mise en application concrète de compétences techniques dans un domaine d'actualité.

Résumé

Face à l'évolution constante des menaces informatiques, la détection d'intrusion constitue un enjeu majeur pour la sécurité des réseaux. Ce travail propose une approche fondée sur l'apprentissage automatique pour construire un système capable de différencier les comportements normaux des activités malveillantes. À partir du jeu de données NSL-KDD, nous avons appliqué plusieurs modèles de classification, dont SVM, Random Forest et XGBoost, en évaluant leurs performances selon des indicateurs standards. L'ensemble du processus inclut le nettoyage des données, leur transformation, l'entraînement des modèles et la mise en place d'une interface utilisateur simplifiée permettant de charger des données et d'obtenir une prédiction en temps réel. L'étude comparative a montré que Random Forest et XGBoost sont les modèles les plus performants dans notre contexte, Random Forest étant retenu pour l'interface. Ce projet ouvre la voie à de futures améliorations, notamment à travers l'utilisation de bases de données plus complètes comme CICIDS2017 et l'exploration de classifications multi-classes.

Abstract

In response to the continuous evolution of cyber threats, intrusion detection has become a key challenge in securing network systems. This project explores a machine learning-based approach to build a system capable of distinguishing between normal and malicious network activities. Using the NSL-KDD dataset, several classification algorithms—namely SVM, Random Forest, and XGBoost—were implemented and evaluated through standard performance metrics. The process involved data cleaning, transformation, model training, and the development of a simple user interface to upload data and generate real-time predictions. The comparative analysis revealed that Random Forest and XGBoost outperformed SVM in this context, and Random Forest was retained for the user interface. This work paves the way for further improvements, particularly through the integration of more comprehensive datasets like CICIDS2017 and the adoption of multi-class classification to better identify specific attack types.

Table des matières

I État de l'art **10**

1 Le système de détection d'intrusions **11**
1.1 Introduction . . . 11
1.2 Définition . . . 11
1.3 Architecture des IDS . . . 12
1.4 Catégories des IDS . . . 13
1.4.1 La détection d'intrusion basée sur les hôtes (HIDS) . . . 13
1.4.2 La détection d'intrusion basée sur le réseau NIDS . . . 14
1.5 Approches conventionnelles de détection . . . 15
1.5.1 Détection basée sur les signatures . . . 15
1.5.2 Détection basée sur les anomalies . . . 16
1.6 Limites des approches traditionnelles . . . 17
1.7 Conclusion . . . 17

2 Machine Learning **18**
2.1 Introduction . . . 18
2.2 Définition de l'intelligence artificielle . . . 18
2.3 Définition du Machine Learning . . . 19
2.4 Apport du Machine Learning dans la détection d'intrusions . . . 19
2.5 Les types du Machine Learning . . . 19
2.5.1 L'apprentissage supervisé . . . 20
2.5.2 L'apprentissage non supervisé . . . 21
2.6 Les algorithmes de Machine Learning . . . 22
2.6.1 SVM . . . 22
2.6.2 Arbre de décision . . . 22
2.6.3 Random Forest . . . 23
2.6.4 XGBoost . . . 24
2.7 Conclusion . . . 25

II Conception et réalisation **26**

3 Approche proposée **27**
3.1 Introduction . . . 27
3.2 Approche détaillée . . . 27
3.2.1 Architecture du modèle . . . 27
3.2.2 Description du dataset . . . 28

3.3 Réalisation de classification du modèle 29
3.4 Prétraitement des données 30
3.4.1 Nettoyage des données 30
3.4.2 Encodage 31
3.4.3 Séparation des données 31
3.5 Implémentation des algorithmes de Machine Learning 32
3.5.1 Rapport de classification 32
3.5.2 Matrice de confusion 32
3.5.3 Courbe ROC 33
3.5.4 Courbe d'apprentissage 33
3.5.5 SVM 33
3.5.6 Random Forest 36
3.5.7 XGBoost 39
3.6 Analyse des résultats 42
3.7 Conclusion 42

4 Intégration d'une interface utilisateur de prédiction 44
4.1 Introduction 44
4.2 Structure générale 44
4.3 Conclusion 48

Bibliographie 59

Table des figures

1.1 Architecture classique d'un système de détection d'intrusion [5] 12
1.2 Système de détection d'intrusion basé sur les hôtes HIDS 14
1.3 Système de détection d'intrusion basé sur le réseau NIDS 15
1.4 Illustration de l'approche par signature [7] 16
1.5 Illustration de l'approche comportementale [8] 16

2.1 Représentation des différents types d'intelligence artificielle 19
2.2 Les types de Machine Learning 20
2.3 La classification et la régression 21
2.4 Support Vector Machine (SVM) 22
2.5 Exemple de données utilisées pour générer un arbre de décision 23
2.6 Arbre de décision généré à partir des données précédentes 23
2.7 Principe de fonctionnement d'une forêt aléatoire 24
2.8 Exemple illustrant l'amélioration des prédictions par XGBoost 25

3.1 Architecture générale du système de détection d'intrusion 28
3.2 Les étapes du prétraitement des données 30
3.3 Nettoyage des données et regroupement des étiquettes 31
3.4 Encodage one-hot et alignement des colonnes du jeu de test 31
3.5 Séparation des variables explicatives et de l'étiquette 32
3.6 Matrice de confusion [17] 33
3.7 Standardisation des données dans le cadre de l'algorithme SVM 34
3.8 Tableau de rapport de classification de l'algorithme SVM 34
3.9 Matrice de confusion de l'algorithme SVM 35
3.10 Courbe ROC de l'algorithme SVM 35
3.11 Courbe d'apprentissage de l'algorithme SVM 36
3.12 Tableau de rapport de classification de l'algorithme Random Forest 37
3.13 Matrice de confusion de l'algorithme Random Forest 37
3.14 Courbe ROC de l'algorithme Random Forest 38
3.15 Courbe d'apprentissage de l'algorithme Random Forest 39
3.16 Tableau de rapport de classification de l'algorithme XGBoost 40
3.17 Matrice de confusion de l'algorithme XGBoost 40
3.18 Courbe ROC de l'algorithme XGBoost 41
3.19 Courbe d'apprentissage de l'algorithme XGBoost 41

4.1 Configuration de l'interface et chargement du modèle 45
4.2 Téléverser un fichier CSV 45
4.3 Screenshot de l'accueil de l'interface 46
4.4 Phase de prédiction 46

4.5 Résultat de l'import d'un fichier CSV et aperçu des données 47
4.6 Résumé global et tableau détaillé des prédictions 47
4.7 Graphiques de répartition des prédictions et de distribution des probabilités d'attaque 48
4.8 Répartition globale des prédictions sous forme de graphique en secteurs . . 48
9 Logo de Visual Studio Code 51
10 Logo de Python 52
11 Logo de Jupyter 52
12 Rapport de classification de l'algorithme XGBoost sur CICIDS2017 56
13 Matrice de confusion de l'algorithme XGBoost sur CICIDS2017 57
14 Courbe ROC de l'algorithme XGBoost sur CICIDS2017 57

Liste des tableaux

3.1 Comparaison des modèles de classification pour la détection d'intrusion . . 42

1 La configuration matérielle et logicielle 54

Introduction générale

Avec l'évolution rapide des technologies de l'information et la généralisation de l'usage d'Internet, la sécurité des systèmes informatiques est devenue un enjeu majeur. Le libre accès aux ressources en ligne, bien qu'il favorise l'échange d'informations et la communication, expose également les réseaux à de multiples risques et attaques. Pour surveiller ces menaces, les **systèmes de détection d'intrusion (IDS)** jouent un rôle crucial, en analysant le trafic pour distinguer les comportements légitimes des activités malveillantes.

Depuis plusieurs années, de nombreuses recherches se sont concentrées sur l'amélioration de ces systèmes. Pourtant, des défis persistent : certaines attaques présentent des structures proches du trafic normal, ce qui rend leur détection difficile, et les performances des modèles existants doivent être renforcées pour améliorer la précision et la réactivité. C'est dans ce contexte que l'**intelligence artificielle**, et plus particulièrement **le Machine Learning**, s'impose comme une solution efficace pour automatiser l'analyse et renforcer la détection des anomalies.

Le principal objectif de ce travail est de **concevoir un système de détection d'intrusion performant** basé sur des algorithmes de Machine Learning. Il s'agit non seulement de classifier le trafic réseau entre activités normales et intrusions, mais aussi de réduire l'impact des attributs inutiles et de comparer plusieurs modèles afin d'identifier celui offrant les meilleurs résultats. Ce système doit permettre d'évaluer les attaques à travers des métriques fiables telles que la précision, le rappel ou l'AUC, et d'aboutir à une implémentation concrète à l'aide d'une interface simple à utiliser.

Afin de répondre à cet objectif, ce rapport est structuré de la façon suivante :

La première partie, à vocation théorique, comprend deux chapitres :

- Le premier chapitre présente les bases des systèmes de détection d'intrusions (IDS), leurs types et leur fonctionnement.
- Le second chapitre est consacré à l'introduction du machine learning dans ce domaine, en abordant les techniques d'apprentissage supervisé, la sélection de caractéristiques, ainsi que les algorithmes les plus utilisés dans le contexte de la cybersécurité.

La deuxième partie est dédiée à la réalisation pratique du projet. Elle se compose de deux chapitres :

- Le premier chapitre décrit l'approche méthodologique adoptée pour concevoir notre système.
- Le deuxième chapitre présente l'implémentation de l'interface utilisateur de prédiction.

Première partie

État de l'art

Chapitre 1

Le système de détection d'intrusions

1.1 Introduction

Avec l'essor fulgurant des technologies réseau, notamment celles reposant sur les connexions sans fil, la sécurisation de ces réseaux est devenue une priorité face aux menaces, qu'elles soient accidentelles ou délibérées.

Toutes les données échangées via Internet ou stockées dans des bases de données doivent impérativement être sécurisées. Les intrusions, qu'elles prennent la forme d'accès non autorisés ou d'abus malveillants des ressources informatiques, représentent un véritable danger en violant les politiques de sécurité en place.

Les solutions de sécurité classiques telles que les pares-feux ou le chiffrement ne suffisent plus à contrer les attaques modernes, de plus en plus complexes et évolutives.

Dès lors, relever les défis de la cybersécurité, détecter les intrusions et garantir la confidentialité des informations s'impose comme une problématique essentielle.

C'est dans ce contexte que James E. Anderson a introduit en 1980 le concept de détection d'intrusion, visant à repérer toute activité suspecte au sein d'un réseau [1].

Ce chapitre propose une vue d'ensemble des systèmes de détection d'intrusions (IDS), en exposant tout d'abord leur définition et leurs types. Par la suite, une classification détaillée des IDS est présentée, ainsi qu'une analyse des différentes approches de détection et des critères d'évaluation permettant de mesurer l'efficacité de ces systèmes.

1.2 Définition

Une intrusion désigne un accès non autorisé à un système informatique, souvent accompagné d'altérations ou de dégradations des données qu'il contient [2].

Le tout premier système de détection d'intrusion a été mis au point en 1984 par Dorothy Denning et Peter Neuman. Ce modèle initial reposait sur une approche comportementale définie par un ensemble de règles. Ce système fut baptisé IDES.

Un **IDS (Intrusion Detection System)** désigne un dispositif, logiciel ou matériel qui joue un rôle essentiel dans l'analyse et la surveillance des flux de données circulant dans un environnement informatique, afin de détecter d'éventuelles tentatives d'intrusion [3].

En effet, lorsqu'une activité sortant du cadre normal est détectée, une alerte est transmise à l'administrateur, qui peut alors décider d'intervenir pour bloquer ou autoriser l'action.

Pourquoi utilise-t-on les IDS ?

Un système de détection d'intrusion (IDS) joue un rôle clé dans la sécurité informatique en collectant des informations précieuses sur les attaques ou tentatives d'intrusion, facilitant ainsi l'analyse, la résolution des incidents et la correction des vulnérabilités.

Il contribue au pilotage de la sécurité des systèmes d'information, en particulier dans les grandes structures, en identifiant les objectifs des attaques et en renforçant les dispositifs existants.

De plus, il permet de détecter des menaces que d'autres mécanismes de sécurité ne parviennent pas toujours à anticiper, tout en dissuadant les comportements malveillants [4].

1.3 Architecture des IDS

Tout d'abord, nous décrivons, dans cette section, les composants d'un système de détection d'intrusion en citant leurs caractéristiques.

Nous présentons cette figure ci-dessous qui les illustre :

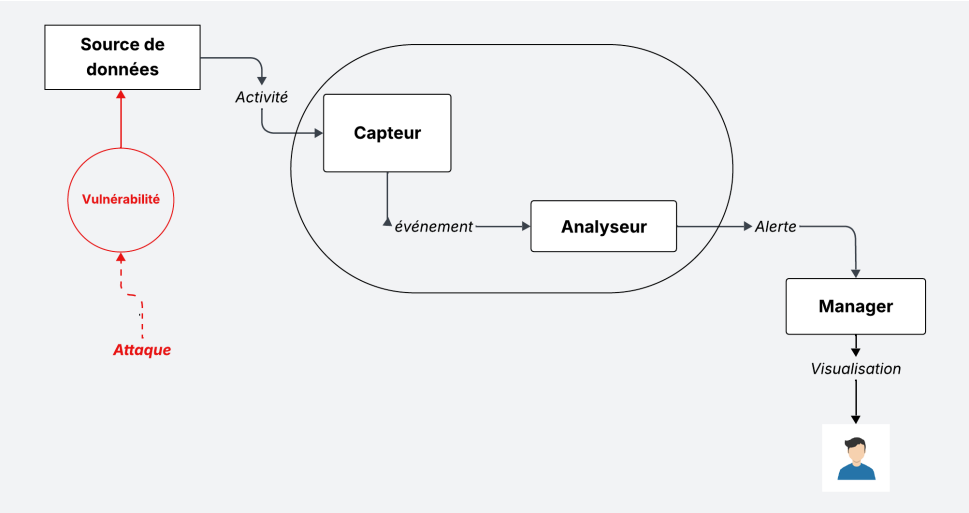


FIGURE 1.1 – Architecture classique d'un système de détection d'intrusion [5]

— **Source de données** : il s'agit d'un mécanisme fournissant des informations sur les activités des différentes entités du système d'information. Ces informations peuvent provenir de diverses origines, comme le réseau, les systèmes, les applications ou encore les alertes de sécurité.

— **Capteur** : il est un composant logiciel essentiel d'un système de détection d'intrusions, chargé de surveiller les activités du système ou du réseau. Il collecte des données brutes, puis les filtre, les structure et les transforme en événements exploitables, qu'il transmet ensuite à un analyseur pour une évaluation approfondie. Bien qu'il puisse se contenter de transmettre les données telles quelles, en général un prétraitement est réalisé.

— **Analyseur** : c'est un outil logiciel qui examine les flux d'événements générés par les capteurs afin de repérer toute activité potentiellement malveillante. Lorsqu'une menace est détectée, une alerte est émise.

— **Alerte** : il s'agit d'une notification formatée envoyée par l'analyseur lorsqu'une activité suspecte est détectée dans les données.

— **Manager** : c'est un composant central d'un système de détection d'intrusions chargé de regrouper les alertes générées par les capteurs, de les organiser et de les présenter à l'opérateur. Il peut également être responsable de la réponse à apporter, qui peut inclure :

1. Isoler l'attaque pour limiter l'impact de l'attaque en cours.
2. Supprimer l'attaque pour l'arrêter.
3. Le recouvrement, qui permet de restaurer le système à un état sûr et fonctionnel.
4. Le diagnostic, qui est une étape d'analyse visant à comprendre la nature et l'origine du problème.

1.4 Catégories des IDS

D'après [6], plusieurs recherches ont été lancées concernant les techniques et les méthodes de détection des intrusions. Comme résultat, on peut classifier les systèmes de détection d'intrusion selon leurs sources de données. Dans cette partie, nous découvrons les classifications les plus communes des IDS : les IDS basés sur les hôtes (HIDS) et les IDS basés sur le réseau (NIDS).

1.4.1 La détection d'intrusion basée sur les hôtes (HIDS)

Le système de détection d'intrusion basé sur l'hôte HIDS permet d'identifier les activités suspectes sur une machine spécifique. Généralement, cela consiste à installer un agent logiciel directement sur l'ordinateur concerné, qui surveille en permanence les activités des applications. Le HIDS contrôle les accès au système, vérifie les comportements des programmes et signale toute action inhabituelle. Il observe en continu divers journaux tels que ceux du système, des applications, des événements, et surveille également l'intégrité des fichiers, et le respect des politiques utilisateurs. Ce système établit une référence des comportements normaux (baseline) et compare chaque nouvelle entrée de journal à cette référence. Si une anomalie est détectée, une alerte est générée. En fonction de la politique configurée, le HIDS peut simplement avertir l'utilisateur, bloquer l'activité suspecte, ou appliquer une autre réponse prévue.

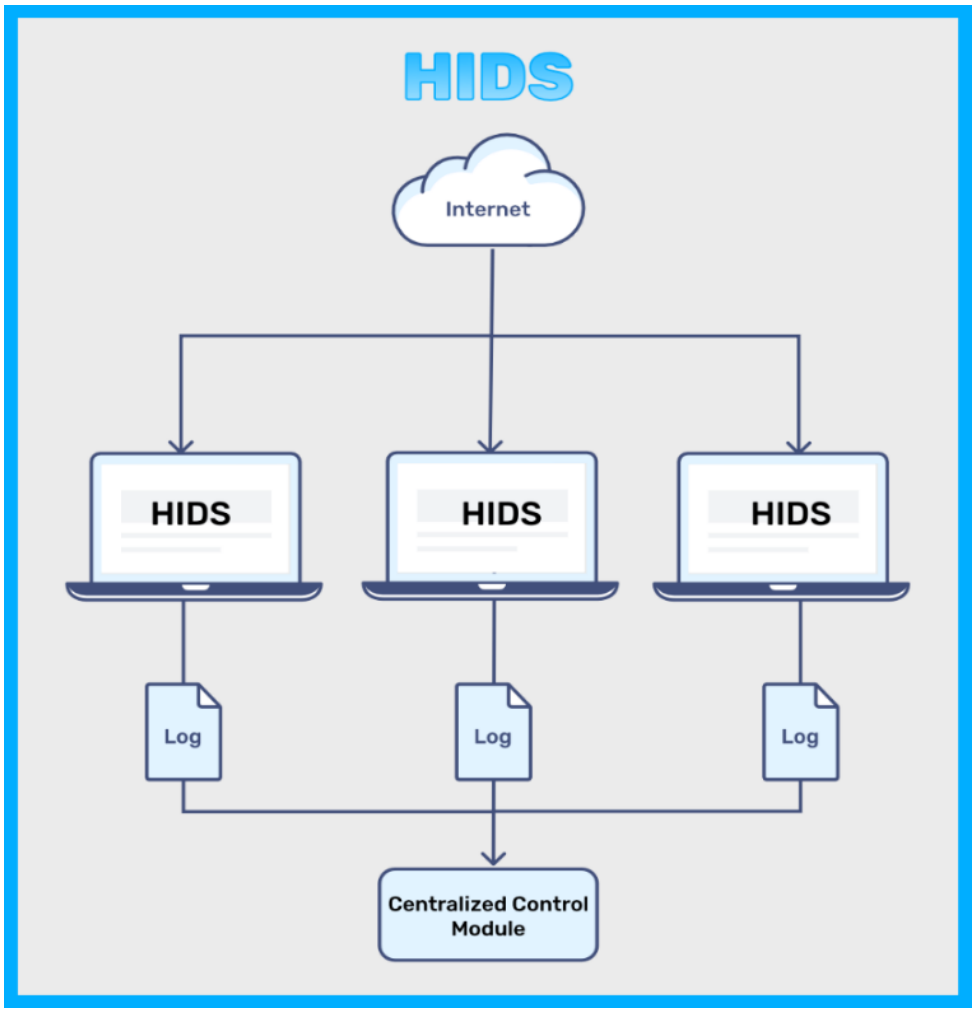


FIGURE 1.2 – Système de détection d'intrusion basé sur les hôtes HIDS

1.4.2 La détection d'intrusion basée sur le réseau NIDS

Un **système de détection d'intrusion réseau NIDS** est un mécanisme essentiel pour surveiller le trafic réseau et analyser les paquets circulant afin de détecter les comportements suspects ou malveillants. Lorsqu'une activité anormale est identifiée ou qu'une attaque est repérée, le NIDS peut générer une alerte destinée à l'administrateur afin qu'il prenne les mesures appropriées. Les NIDS sont conçus pour détecter quatre principaux types d'attaques : le déni de service (DoS), le scan de vulnérabilités (Probe), les attaques utilisateur vers root (U2R) et les attaques à distance vers utilisateur (R2L). Ces systèmes sont particulièrement adaptés aux environnements multi-plateformes et offrent une gestion centralisée, ce qui facilite le contrôle des infrastructures complexes.

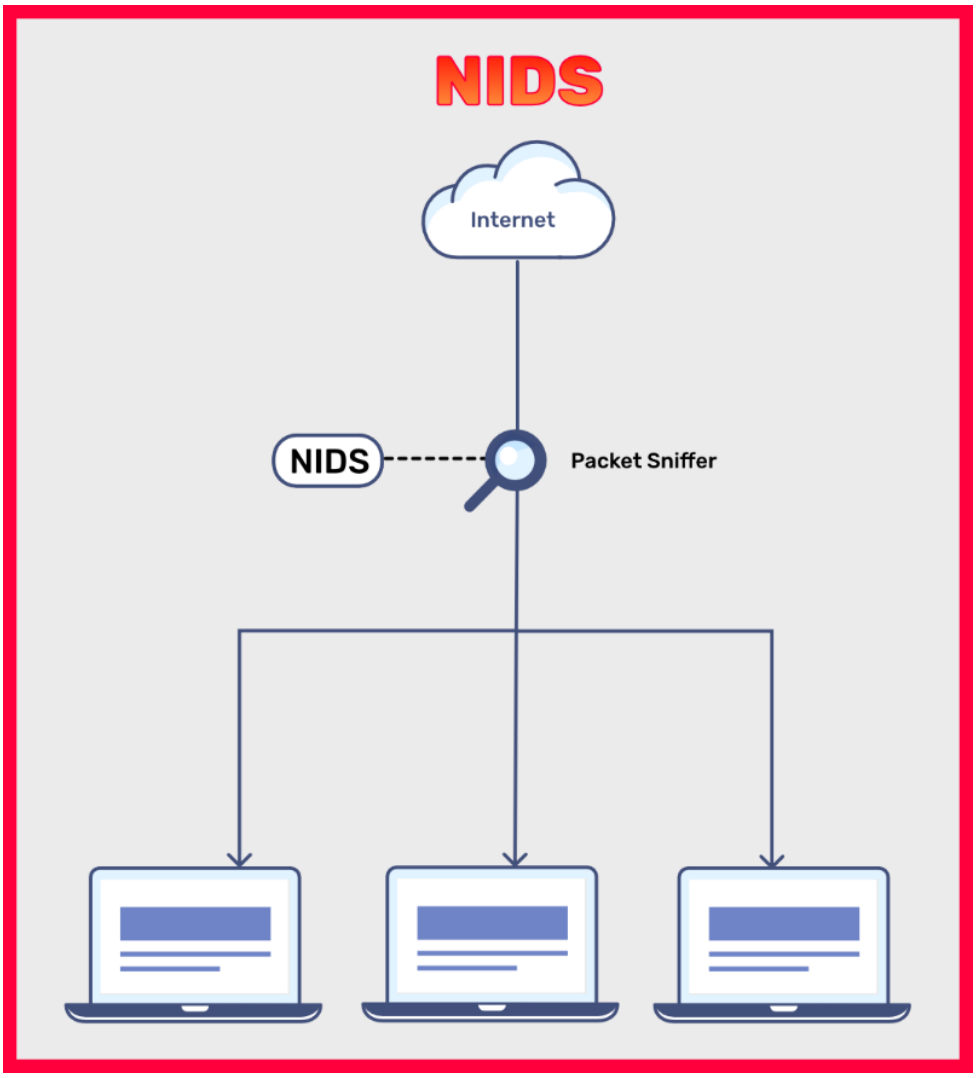


FIGURE 1.3 – Système de détection d'intrusion basé sur le réseau NIDS

1.5 Approches conventionnelles de détection

1.5.1 Détection basée sur les signatures

Une **signature** est un modèle qui correspond à une menace connue. **La détection basée sur les signatures** consiste à comparer des signatures prédéfinies avec des événements observés afin d'identifier des incidents potentiels.

Bien que la détection par signatures soit efficace pour repérer les menaces connues, elle est inefficace face aux menaces inconnues, aux techniques d'évasion ou aux variantes existantes.

Cette méthode est simple : elle compare l'activité courante à une liste de signatures via des comparaisons de chaînes. Toutefois, elle ne comprend pas les protocoles ni les applications et ne peut pas suivre des sessions complexes, comme l'enchaînement requête-réponse HTTP (ex : erreur **403** non détectée). Elle ne garde pas non plus la mémoire des requêtes passées, ce qui rend difficile la détection d'attaques réparties.

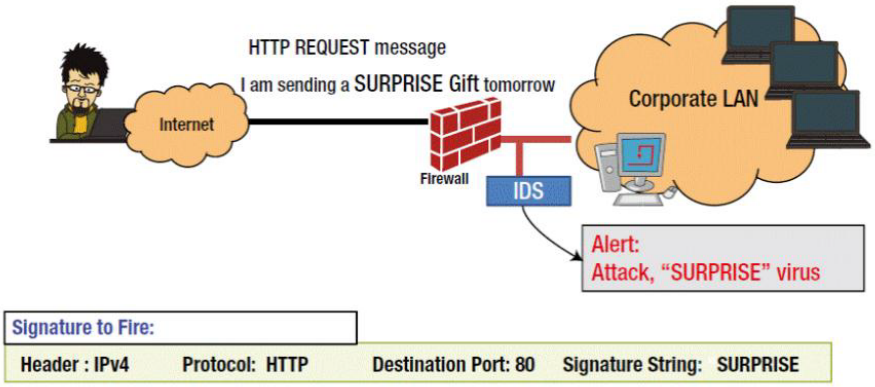


FIGURE 1.4 – Illustration de l'approche par signature [7]

1.5.2 Détection basée sur les anomalies

La détection basée sur les anomalies repose sur l'idée que tout comportement déviant de la norme est potentiellement malveillant : **« Si ce n'est pas normal, c'est potentiellement dangereux »**.

Cette approche construit un **modèle comportemental** (baseline) basé sur des paramètres du système ou des utilisateurs :

- **Charge CPU** : pics d'utilisation anormaux.
- **Volume de données échangées** : variations inhabituelles.
- **Durée et heure des connexions** : accès tardifs ou prolongés.
- **Protocoles et applications** : usage anormal (ex : FTP dans un environnement HTTP).

Dès qu'une activité dévie de la baseline, elle est signalée comme suspecte. Cependant, des faux positifs peuvent survenir si la baseline n'est pas bien ajustée. Il est donc crucial de la mettre à jour régulièrement.

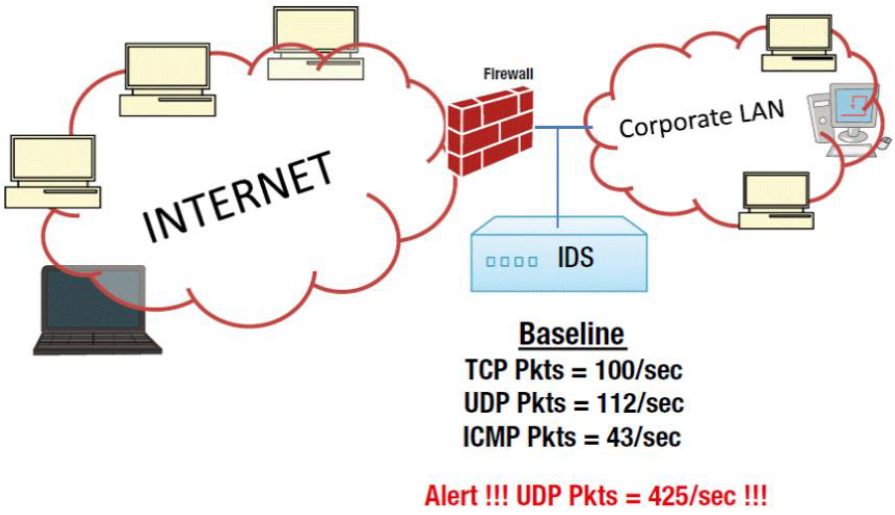


FIGURE 1.5 – Illustration de l'approche comportementale [8]

1.6 Limites des approches traditionnelles

Les méthodes traditionnelles de détection d'intrusion présentent plusieurs limites majeures. Tout d'abord, les approches basées sur les signatures ne peuvent identifier que les attaques déjà connues ; toute menace nouvelle ou modifiée échappe ainsi à leur détection. De plus, leur efficacité repose sur des mises à jour fréquentes de la base de signatures, ce qui les rend dépendantes d'un suivi constant.

Les méthodes statistiques ou issues du *data mining*, quant à elles, bien qu'efficaces pour repérer des comportements inhabituels, exigent une forte puissance de calcul, notamment lors de la phase de création du modèle, et génèrent souvent de nombreux faux positifs, en particulier dans les environnements complexes et changeants.

Par ailleurs, les systèmes de détection basés sur le réseau **NIDS** ne sont pas en mesure de repérer les attaques locales initiées directement depuis la machine cible, car ils se contentent d'analyser le trafic réseau sans accès aux événements internes de l'hôte. De leur côté, les systèmes basés sur l'hôte **HIDS**, bien qu'ayant une visibilité complète sur la machine, deviennent peu fiables une fois que celle-ci est compromise, car les données collectées peuvent être altérées ou manipulées [9].

1.7 Conclusion

Les systèmes de détection d'intrusion, qu'ils soient déployés au niveau des hôtes HIDS ou du réseau NIDS, ont longtemps constitué une base essentielle pour protéger les environnements informatiques. Basés sur des techniques de détection par signature ou d'analyse des anomalies, ces systèmes surveillent les activités et signalent tout comportement suspect.

Cependant, malgré leur efficacité contre les menaces déjà identifiées, ces méthodes présentent aujourd'hui des limites notables : elles peinent à détecter des attaques inconnues, nécessitent des mises à jour régulières, consomment beaucoup de ressources et génèrent souvent des faux positifs, notamment dans des environnements complexes.

Ces lacunes mettent en évidence la nécessité d'adopter des solutions plus avancées, capables de s'adapter aux nouvelles menaces de manière dynamique. C'est dans ce contexte que les techniques basées sur le Machine Learning apparaissent comme des alternatives prometteuses. En exploitant l'analyse des données et des modèles prédictifs, ces approches permettent non seulement d'améliorer la détection des attaques mais aussi de prévoir des comportements malveillants avant même qu'ils ne surviennent.

La section suivante explorera en détail les principes du Machine Learning appliqués aux IDS, ses avantages, ainsi que les méthodes couramment utilisées.

Chapitre 2

Machine Learning

2.1 Introduction

Dans le cadre des systèmes de détection d'intrusion, le concept de classification joue un rôle central en permettant de distinguer et d'identifier les attaques potentielles. Afin de faciliter l'analyse des incidents et de structurer les informations issues des journaux d'audit, certains chercheurs ont proposé de catégoriser les attaques pour mieux les gérer. Cependant, bien que cette approche ait largement contribué à l'adoption des IDS, elle soulève également des défis majeurs pour l'évaluation de ces systèmes, notamment en termes de précision et d'efficacité.

Pour aborder ces problématiques, il est essentiel de commencer par une analyse approfondie des algorithmes de classification sélectionnés et d'examiner leur application spécifique dans le contexte de la détection d'intrusion.

L'**apprentissage automatique (Machine Learning)** permet à un système d'apprendre à partir de données plutôt que d'être programmé de manière rigide. Aujourd'hui, le Machine Learning est omniprésent et trouve des applications variées dans de nombreux secteurs. L'objectif fondamental du Machine learning est de former un modèle capable de traiter efficacement les données disponibles afin d'en extraire des informations pertinentes. Ce processus se décompose généralement en deux phases principales : **l'apprentissage**, où le modèle est entraîné à partir des données d'entraînement, et **la prédiction**, où le modèle est utilisé pour générer des prévisions basées sur de nouvelles données.

Dans ce chapitre, nous présenterons l'apprentissage automatique et ses types, ainsi que les principales séries d'algorithmes pour chaque type.

2.2 Définition de l'intelligence artificielle

L'intelligence artificielle IA est une branche des sciences informatiques qui vise à concevoir des systèmes capables d'exécuter des tâches généralement associées à l'intelligence humaine. Selon Marvin Minsky, l'un des pionniers du domaine, l'IA consiste à « faire en sorte que des machines accomplissent des actions qui, si elles étaient réalisées par un humain, nécessiteraient de l'intelligence ».

Cette discipline repose sur des fondements mathématiques et s'appuie sur des algorithmes exploitant les probabilités et les statistiques afin de simuler des processus cognitifs tels que l'apprentissage, le raisonnement ou encore la prise de décision.

L'intelligence artificielle se décline principalement en deux sous-domaines : le **Machine Learning**, qui permet aux machines d'apprendre automatiquement à partir de données,

et le **Deep Learning**, qui utilise des réseaux de neurones artificiels inspirés du cerveau humain pour analyser de grandes quantités d'informations complexes. [10].

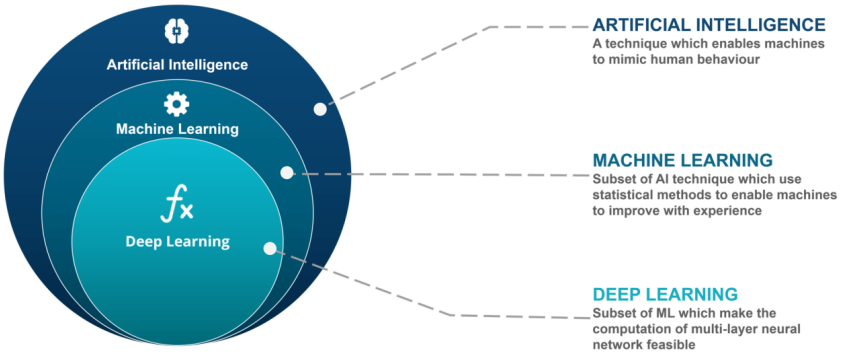


FIGURE 2.1 – Représentation des différents types d'intelligence artificielle

2.3 Définition du Machine Learning

Le **Machine Learning**, ou l'**apprentissage automatique**, est un concept de plus en plus présent dans le domaine informatique, étroitement lié à l'intelligence artificielle. Il s'agit d'un processus reposant sur l'analyse et la modélisation de données dans le but de mettre en place des mécanismes capables d'apprendre sans intervention humaine explicite.

Concrètement, cela consiste à développer des programmes permettant à une machine ou un ordinateur d'apprendre automatiquement à partir d'un grand volume d'informations. Cette capacité d'apprentissage autonome permet d'exécuter des tâches complexes, souvent impossibles à résoudre par des approches traditionnelles.

L'objectif est de doter les machines d'analyser des données massives, de détecter des relations ou des schémas récurrents entre différentes variables, et de formuler des prédictions sur la base de ces observations. Grâce à cette approche, il devient possible d'anticiper des résultats ou des comportements dans des contextes variés [11].

2.4 Apport du Machine Learning dans la détection d'intrusions

Le Machine Learning joue un rôle essentiel dans l'évolution des systèmes de détection d'intrusion. Contrairement aux approches classiques basées sur des règles fixes ou des signatures prédéfinies, l'apprentissage automatique permet aux systèmes de reconnaître des schémas complexes dans les données réseau et de détecter des comportements anormaux, y compris ceux qui n'ont jamais été observés auparavant.

Grâce à l'analyse de volumes massifs de données, les algorithmes de machine learning sont capables de s'adapter à de nouveaux types d'attaques, d'améliorer continuellement leur précision au fil du temps et de réduire les faux positifs.

2.5 Les types du Machine Learning

Dans le domaine du **Machine Learning**, il existe deux principaux types d'apprentissages : **supervisé** et **non supervisé**. La principale différence entre ces deux types est que l'ap-

prentissage supervisé se fait sur la base d'un fait. En d'autres termes, nous avons une connaissance préalable de ce que devrait être la valeur de sortie de l'échantillon. En revanche, l'apprentissage non supervisé n'a pas de résultats étiquetés. Son objectif est de dériver la structure naturelle qui existe dans un ensemble de points de données.

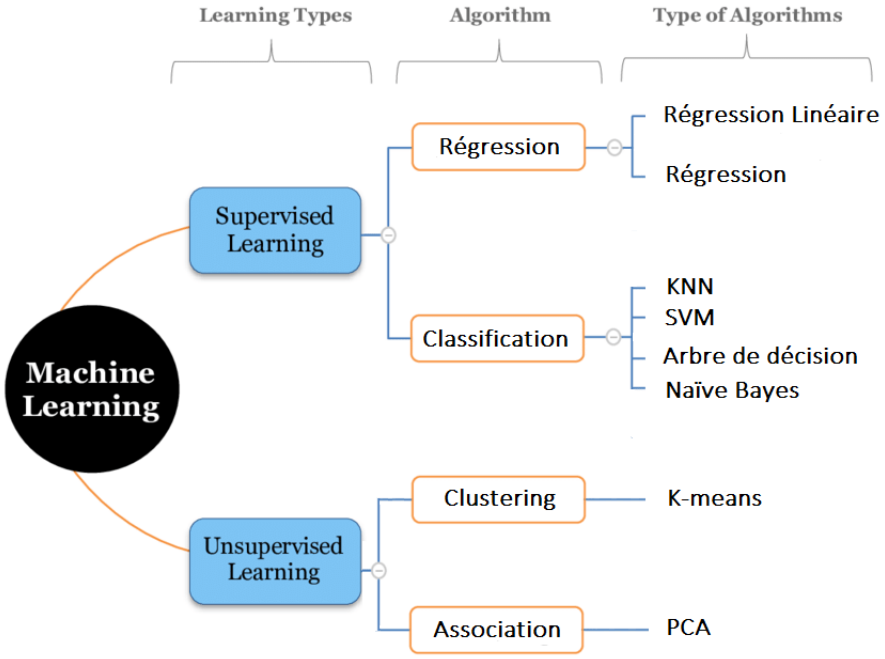


FIGURE 2.2 – Les types de Machine Learning

2.5.1 L'apprentissage supervisé

La plupart des approches en Machine Learning reposent sur le principe de **l'apprentissage supervisé**. Cette méthode s'appuie sur l'utilisation d'un ensemble de données comportant à la fois des **valeurs d'entrée (X)** et leurs **résultats associés (Y)**. L'objectif est d'amener un algorithme à apprendre une fonction de correspondance entre les entrées et les sorties, selon la relation : $Y = f(X)$.

L'idée est que, une fois cette fonction bien assimilée, le modèle puisse prédire correctement la sortie Y pour de nouvelles données jamais vues auparavant.

On parle d'apprentissage supervisé car le processus s'apparente à une pédagogie guidée : l'algorithme est formé à partir d'exemples dont les bonnes réponses sont déjà connues. Il réalise des prédictions, reçoit un retour sur ses erreurs, et ajuste progressivement ses paramètres jusqu'à atteindre un niveau de performance satisfaisant.

Ce type d'apprentissage est particulièrement utilisé dans des tâches telles que **la classification** (attribuer une catégorie à une donnée) ou **la régression** (prédire une valeur continue).

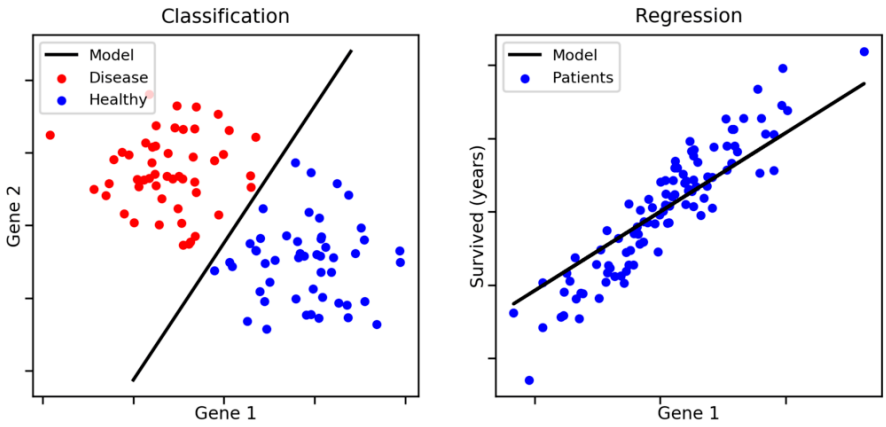


FIGURE 2.3 – La classification et la régression

— **Classification :** Ce type de problème se manifeste lorsque la sortie attendue correspond à une catégorie ou une étiquette. L'objectif est alors de classer les données dans des groupes prédéfinis, comme par exemple : *fraude* ou *non fraude*, *spam* ou *non spam*, ou encore *malade* ou *sain*.

— **Régression :** Contrairement à la classification, la régression traite des **valeurs numériques continues**. Ici, l'objectif est de prédire une quantité mesurable. Par exemple : estimer le prix d'un bien immobilier en fonction de ses caractéristiques ou prédire l'évolution d'un cours boursier sur une période donnée [12].

2.5.2 L'apprentissage non supervisé

L'**apprentissage non supervisé** se caractérise par l'absence de variables cibles. Autrement dit, seules les données d'entrée (**X**) sont disponibles, sans aucune indication sur la sortie attendue. L'objectif principal de cette approche est d'explorer la structure interne des données, de détecter des régularités ou des schémas cachés, et d'en tirer des enseignements sans qu'aucune réponse « correcte » ne soit préalablement définie.

Contrairement à l'apprentissage supervisé, où un « enseignant » guide le modèle en fournissant les résultats attendus, l'apprentissage non supervisé laisse l'algorithme agir de manière autonome pour découvrir des structures pertinentes dans les données. Cette approche est particulièrement utilisée pour l'analyse exploratoire et se divise principalement en deux grandes familles de techniques : le **clustering** (regroupement) et l'**association**.

— **Clustering (Regroupement)** : Cette méthode consiste à diviser un ensemble de données en sous-groupes appelés *clusters*, de manière à ce que les données au sein d'un même groupe soient plus similaires entre elles qu'avec celles des autres groupes.

— **Association** : Cette technique vise à mettre en évidence des relations fréquentes entre différents éléments présents dans de vastes ensembles de données. Les règles d'association permettent donc de prévoir quelles combinaisons d'éléments apparaissent souvent ensemble, facilitant ainsi les recommandations ou les analyses de comportement d'achat [12].

2.6 Les algorithmes de Machine Learning

2.6.1 SVM

Le **Support Vector Machine (SVM)**, est une technique utilisée aussi bien pour des **tâches de classification**, de **régression** que pour la détection d'anomalies. Introduits dans les années 1990, les SVM sont appréciés pour leur robustesse théorique.

Le principe de fonctionnement des SVM est relativement intuitif : il s'agit de trouver une frontière optimale qui sépare au mieux les différentes classes de données. Cette frontière, appelée hyperplan, est choisie de manière à maximiser la distance entre cette séparation et les points les plus proches appartenant à chaque classe. Plus cette marge est large, plus la séparation est considérée comme fiable. Les vecteurs de support sont justement ces points situés au bord de chaque groupe, qui définissent la position de l'hyperplan [13]

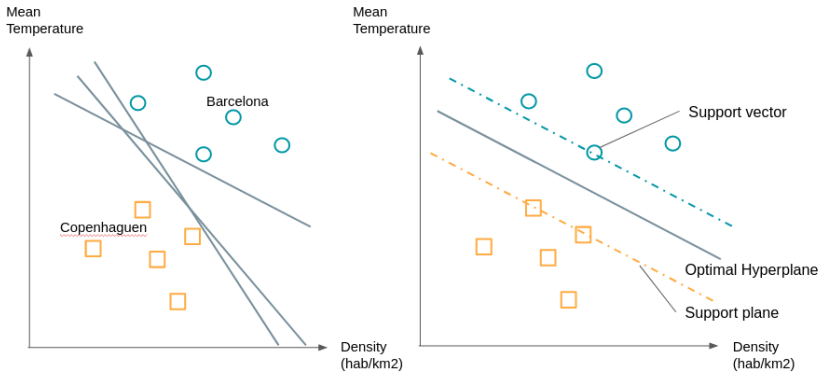


FIGURE 2.4 – Support Vector Machine (SVM)

2.6.2 Arbre de décision

L'**arbre de décision** est un algorithme de classification largement utilisé dans divers secteurs. Il repose sur une représentation graphique arborescente, où chaque chemin correspond à une série de décisions aboutissant à une conclusion, tout en prenant en compte les probabilités associées aux différents scénarios.

Concrètement, à partir d'un ensemble de données étiquetées, l'arbre permet de prédire la classe à laquelle appartiendra une nouvelle observation.

Le parcours de l'arbre consiste à suivre ces règles successives, depuis la racine jusqu'à une feuille, ce qui permet d'affecter une classe à une observation en fonction de ses caractéristiques. L'arbre est structuré de manière à optimiser chaque division, en sélectionnant à chaque étape la règle qui sépare le mieux les observations selon leur étiquette.

— **Exemple** :

— **Les données** :

Observation	Petal length	Stem length	Species
1	2.7 cm	20 cm	A
2	2.6 cm	12 cm	C
3	2.1 cm	21 cm	B
4	1.9 cm	20 cm	B

Data

FIGURE 2.5 – Exemple de données utilisées pour générer un arbre de décision

— **L'arbre** :

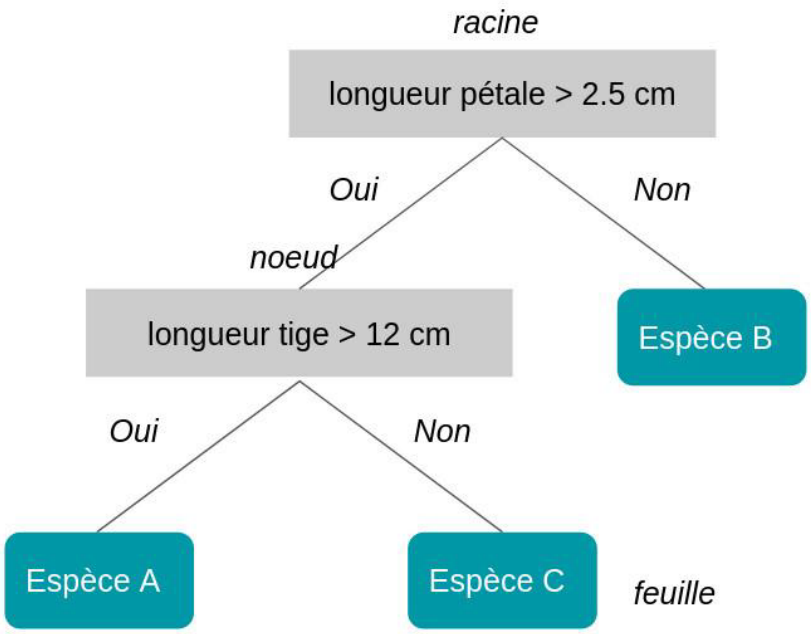


FIGURE 2.6 – Arbre de décision généré à partir des données précédentes

La **profondeur de l'arbre** correspond au nombre de décisions nécessaires avant d'atteindre une prédiction. Par exemple, un arbre de profondeur 2 signifie qu'il faut au maximum deux tests pour aboutir à une décision finale [13]

2.6.3 Random Forest

L'algorithme **Random Forest** repose sur le principe des **arbres de décision**, mais introduit un mécanisme collectif pour améliorer la précision et éviter le surapprentissage.

Pour illustrer son fonctionnement, prenons un exemple simple : vous cherchez une destination pour vos prochaines vacances. Vous commencez par demander conseil à vos

amis. Chacun vous interroge à sa façon, en sélectionnant de manière aléatoire des critères différents (climat, budget, culture, etc.), puis vous donne une recommandation. Cette approche collective est justement ce qui caractérise une forêt aléatoire : un ensemble d'arbres de décision qui, combinés, donnent une réponse plus fiable et moins sujette à l'*overfitting* (le fait qu'un modèle soit trop ajusté à des données particulières) [13]

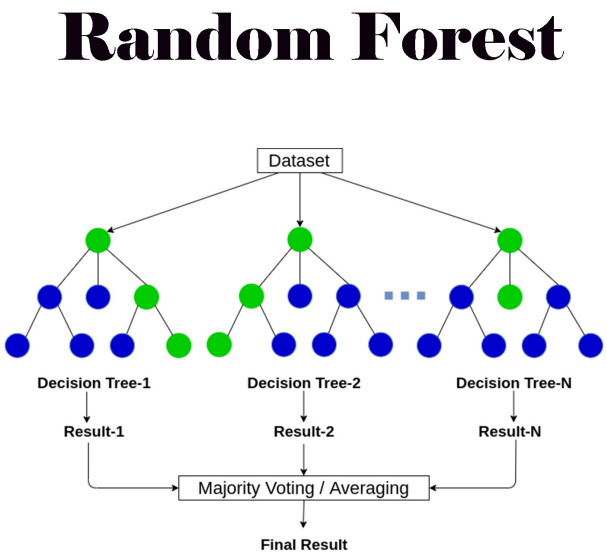


FIGURE 2.7 – Principe de fonctionnement d'une forêt aléatoire

2.6.4 XGBoost

La technique du **gradient boosting** vise à améliorer la performance d'un modèle de base, souvent simple comme un arbre de décision, en corrigeant ses erreurs progressivement. Cette méthode repose sur l'idée d'enchaîner plusieurs modèles faibles afin de construire un prédicteur plus puissant.

Prenons un exemple concret : vous disposez d'un jeu de données contenant des informations sur les activités et la démographie d'un groupe d'individus. Pour la moitié d'entre eux, l'âge est connu, pour l'autre non. Votre objectif est de prédire l'âge en fonction de comportements comme les courses, les loisirs, ou le temps passé devant un écran. Vous choisissez pour cela un arbre de régression, car la variable à estimer est continue.

Votre premier modèle fournit des résultats raisonnables, mais comporte encore de larges écarts entre les valeurs réelles et celles prédites. Par exemple, une personne âgée de 13 ans est estimée à 19 ans, et une autre de 68 ans est prédite à 55 ans.

Le gradient boosting intervient ici : au lieu de reconstruire un modèle complet, on va créer un nouveau modèle qui tente de prédire uniquement les erreurs du précédent. Ce second modèle apprend donc à corriger les écarts. Par exemple :

Age	Prediction Tree 1	Difference	Prediction Tree 2
13	19	-6	15
68	55	+13	63

FIGURE 2.8 – Exemple illustrant l'amélioration des prédictions par XGBoost

L'optimisation de ce processus repose sur la descente de gradient, une méthode mathématique permettant d'ajuster les prédictions pour réduire progressivement l'erreur, bien que son fonctionnement détaillé ne soit pas abordé ici.

2.7 Conclusion

Ce chapitre a permis de poser les fondements nécessaires à la compréhension du rôle de l'intelligence artificielle, et plus précisément du Machine Learning, dans le domaine de la détection d'intrusion.

En effet, sa capacité à analyser de grandes quantités de données et à identifier des schémas inhabituels en fait un outil puissant pour détecter aussi bien des attaques connues que des comportements malveillants jusque-là non identifiés.

Nous avons également distingué les différents types d'apprentissage : l'apprentissage supervisé, basé sur des données étiquetées, et l'apprentissage non supervisé, qui explore la structure cachée des données sans connaissance préalable des résultats attendus. Par la suite, nous avons présenté les algorithmes les plus couramment utilisés dans ce domaine, en soulignant les caractéristiques de chacun et leur potentiel à résoudre des problèmes complexes de classification.

Ce cadre théorique nous conduit naturellement vers la partie suivante de notre travail, consacrée à la conception et la mise en œuvre pratique. Nous y détaillerons les choix méthodologiques, les étapes de préparation des données, le développement du modèle, ainsi que l'évaluation des performances du système de détection basé sur les techniques de machine learning [13]

Deuxième partie

Conception et réalisation

Chapitre 3

Approche proposée

3.1 Introduction

Dans ce chapitre, nous nous intéressons à la base de données utilisée dans le cadre de notre projet, ainsi qu'à l'approche méthodologique adoptée pour développer un système de détection d'intrusion efficace. Nous commencerons par une présentation du dataset **NSL-KDD**, en mettant en avant ses caractéristiques, sa structure et son intérêt pour ce type d'application.

Par la suite, nous détaillerons les étapes clés du traitement des données, de leur préparation à leur transformation, ainsi que les modèles de machine learning testés pour la classification. L'objectif est de montrer comment, à partir de données brutes, nous avons construit une démarche complète allant du prétraitement à l'évaluation des performances des différents algorithmes.

3.2 Approche détaillée

3.2.1 Architecture du modèle

En se basant sur le dataset **NSL-KDD**, un système a été développé dans le but de concevoir une solution de détection des intrusions au sein des réseaux informatiques. Ce système commence par une phase de prétraitement appliquée aux données brutes, permettant d'adapter leur format à l'analyse : les étiquettes sont ramenées à deux classes et les attributs symboliques sont convertis en valeurs numériques. Une standardisation des attributs n'est appliquée que pour l'algorithme qui y est sensible (le SVM), au sein de son propre pipeline.

La phase de classification repose sur l'application de trois techniques d'apprentissage automatique (SVM, Random Forest et XGBoost) entraînées sur les mêmes données. Une fois l'entraînement terminé, chaque modèle est mis à l'épreuve en effectuant des prédictions sur les données de test, puis les résultats obtenus sont comparés pour mesurer l'efficacité du système.

L'architecture générale de cette solution est représentée dans la figure suivante :

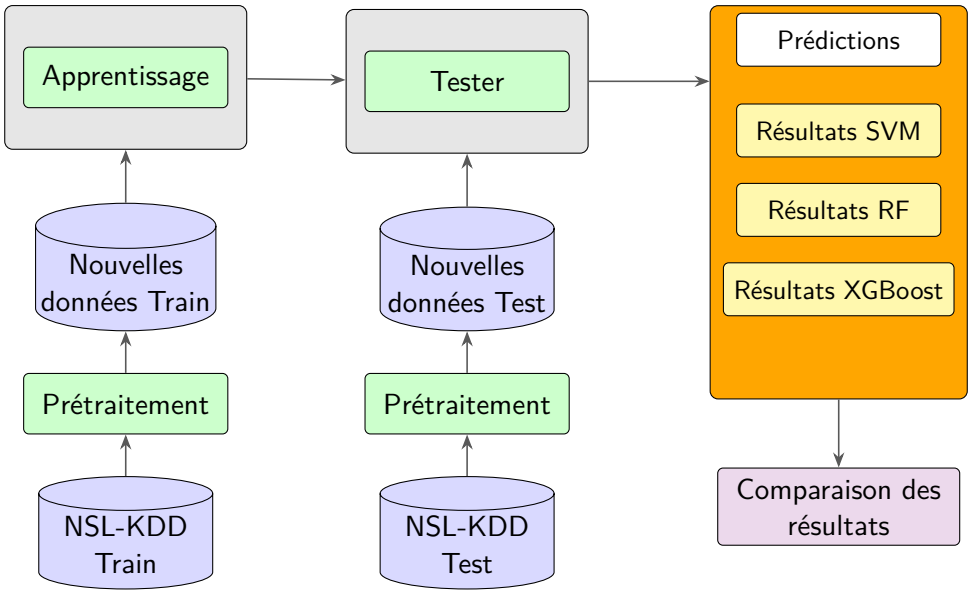


FIGURE 3.1 – Architecture générale du système de détection d'intrusion

3.2.2 Description du dataset

Définition : Le dataset **NSL-KDD** a été créé en 2009 [14]. C'est une version révisée du dataset *KDD Cup 99*, conçue pour évaluer les performances des systèmes de détection d'intrusion. Il comprend **41 attributs** décrivant des caractéristiques réseau, tels que le protocole, le type de service et le volume des paquets, le dernier indiquant si le trafic est **normal** ou constitue une **attaque**. L'étude menée à partir du dataset **NSL-KDD** confirme son efficacité comme base d'évaluation pour tester et comparer les performances de différents modèles de classification. Ainsi, le **NSL-KDD** s'impose comme un jeu de données de référence pour la détection d'intrusions. Sa structure retravaillée, l'élimination des doublons et la meilleure répartition des classes en font un environnement d'évaluation robuste, à la fois pour les chercheurs et pour les systèmes réels de cybersécurité. [15] [16].

Objectifs et améliorations : Cette version a été développée pour corriger certaines lacunes du *KDD Cup 99*, notamment :

- **Réduction des redondances** : le dataset d'entraînement a été filtré pour supprimer les doublons, évitant ainsi le biais qui pourrait favoriser certains modèles lors de l'entraînement.
- **Suppression des doublons dans le jeu de test** : cette modification garantit que chaque instance est unique, permettant une évaluation plus précise des modèles, sans risque de surapprentissage.
- **Rééquilibrage des classes** : les instances ont été sélectionnées pour garantir une distribution plus équilibrée des catégories d'attaques, ce qui améliore la représentativité des échantillons [16].

Structure des données : Le dataset **NSL-KDD** est organisé en deux ensembles principaux :

- **KDDTrain** : **125 973 instances** représentant un large éventail de comportements réseau.
- **KDDTest** : **22 544 instances**, destinées à évaluer la performance des modèles de classification.

Les données sont classées en cinq catégories :

1. **Normal** : Trafic réseau normal.
2. **Dos (Denial of Service)** : Attaques visant à rendre les services indisponibles.
3. **Probe** : Scans visant à identifier les failles de sécurité.
4. **U2R (User to Root)** : Tentatives de prise de contrôle en accédant aux privilèges root.
5. **R2L (Remote to Local)** : Tentatives d'accès à distance non autorisé [16].

Attributs de NSL-KDD : La base de données **NSL-KDD** comprend **41 attributs**, chacun associé à un type de donnée spécifique. Ces attributs peuvent être regroupés en **trois grandes catégories**, selon leur nature et leur rôle dans l'analyse du trafic réseau :

— **Attributs de base** : Ils décrivent les éléments fondamentaux d'une connexion, tels que la *durée de la session*, les *adresses source et destination*, les *ports utilisés*, ainsi que le *type de drapeau TCP* (*flag*).

— **Attributs liés au trafic** : Ces caractéristiques sont issues de *mesures statistiques*, comme le *nombre de connexions établies avec une même machine* dans une fenêtre de temps donnée. Elles permettent de mettre en évidence des comportements répétitifs ou anormaux dans le réseau.

— **Caractéristiques de contenu** : Elles s'appuient sur l'analyse de la *charge utile* (*payload*) des paquets échangés. On y retrouve, par exemple, le *nombre de connexions ayant échoué lors de l'accès à certains fichiers système*, ou les *tentatives d'obtention de privilèges anormales.*

Ces différents groupes d'attributs permettent d'enrichir l'analyse et d'améliorer la capacité des modèles à détecter efficacement les comportements malveillants dans un réseau.

3.3 Réalisation de classification du modèle

Notre projet s'articule autour de trois phases principales : **le prétraitement des données, l'apprentissage du modèle et la phase de test**.

Le processus débute par l'exploitation des deux fichiers fournis avec la base de données : **KDDTrain+** (125 973 connexions) pour l'entraînement et **KDDTest+** (22 544 connexions) pour les tests. Nous avons conservé cette séparation d'origine plutôt que de redécouper nous-mêmes les données, ce qui permet de comparer nos résultats à ceux de la littérature sur le même jeu de test.

Dans un premier temps, les étiquettes sont regroupées en deux classes (*normal* et *attack*), puis une transformation des données symboliques en valeurs numériques est effectuée par encodage one-hot, ce qui porte le nombre d'attributs de 41 à 122. Les colonnes du jeu de test sont ensuite alignées sur celles du jeu d'entraînement. Enfin, on obtient un dataset prêt à être utilisé dans les phases de classification et l'évaluation de sa performance sur des données inconnues.

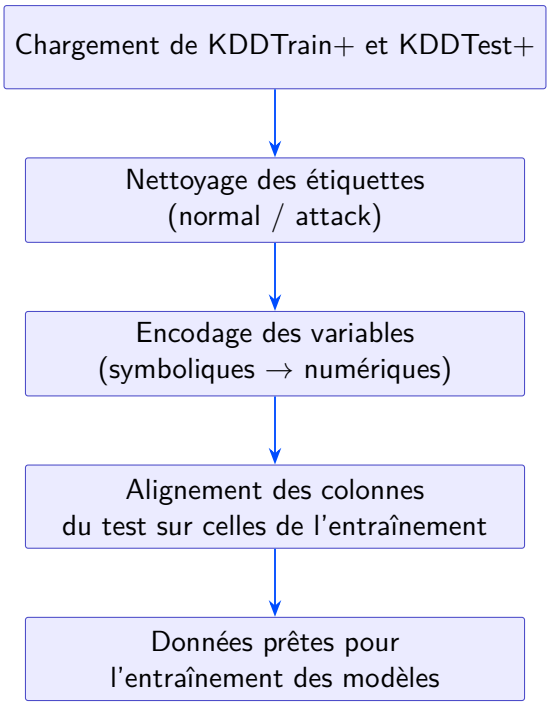


FIGURE 3.2 – Les étapes du prétraitement des données

3.4 Prétraitement des données

Dans cette phase, nous avons préparé les données afin qu'elles puissent être utilisées efficacement par notre modèle. Les étapes principales suivies sont les suivantes :

— **Importation des bibliothèques Python** indispensables pour le traitement, telles que `os`, `pandas`, etc.

— **Chargement des jeux de données** d'entraînement et de test.

La base de données NSL-KDD est accessible via le site Kaggle aux liens suivants :

— **NSL-KDD Train** : `https://www.kaggle.com/datasets/hassan06/nslkdd?select=KDDTrain%2B.arff`

— **NSL-KDD Test** : `https://www.kaggle.com/datasets/hassan06/nslkdd`

3.4.1 Nettoyage des données

Le nettoyage des données est une étape essentielle qui vise à :

— Vérifier le type des colonnes et la structure générale du dataset.

— Regrouper les étiquettes d'origine en deux classes seulement : `normal` et `attack`.

— S'assurer que le format des attributs est adapté à l'analyse.

```python
# 1. Vérifie les types
print("Types des colonnes :")
print(df_train.dtypes.tail())

# 2. Nettoyer les labels : binaire (normal / attack)
df_train['class'] = df_train['class'].apply(lambda x: 'normal' if x == 'normal' else 'attack')
df_test['class'] = df_test['class'].apply(lambda x: 'normal' if x == 'normal' else 'attack')
```

FIGURE 3.3 – Nettoyage des données et regroupement des étiquettes

3.4.2 Encodage

Cette phase consiste à convertir des attributs textuels, tels que `protocol_type`, `service`, et `flag`, en valeurs numériques. Cette transformation est essentielle, car la plupart des algorithmes de Machine Learning ne peuvent pas manipuler directement des données textuelles. L'encodage permet donc de représenter ces informations de manière exploitable tout en conservant leur sens sémantique.

Le jeu de test pouvant ne pas contenir exactement les mêmes modalités que le jeu d'entraînement, ses colonnes sont ensuite alignées sur celles de l'entraînement : une modalité absente est remplie par la valeur 0, et une colonne inconnue du modèle est écartée. Après cette étape, les deux jeux de données comptent 122 attributs, auxquels s'ajoute l'étiquette `class` (conservée à part dans `y_train` et `y_test`).

```python
# 3. Encodage one-hot
cat_cols = ['protocol_type', 'service', 'flag']
df_train = pd.get_dummies(df_train, columns=cat_cols)
df_test = pd.get_dummies(df_test, columns=cat_cols)

# 4. Aligner test sur train
df_test = df_test.reindex(columns=df_train.columns, fill_value=0)
```

FIGURE 3.4 – Encodage one-hot et alignement des colonnes du jeu de test

3.4.3 Séparation des données

Il est indispensable de distinguer les variables explicatives de l'étiquette à prédire, et de conserver deux ensembles distincts :

— **X_train / Y_train** : utilisé pour l'entraînement du modèle.

— **X_test / Y_test** : utilisé pour l'évaluation de ses performances.

Cette démarche permet de produire des prédictions fiables sur des données qu'il n'a jamais vues. En se basant uniquement sur l'ensemble d'entraînement, le modèle risquerait d'apprendre par cœur les données (phénomène connu sous le nom de *surapprentissage*), sans réellement saisir les relations sous-jacentes. Cette séparation garantit une évaluation objective et réaliste, tout en facilitant la comparaison entre plusieurs algorithmes dans des conditions équitables.

```
# 5. Séparer X et y
X_train = df_train.drop('class', axis=1)
y_train = df_train['class']
X_test = df_test.drop('class', axis=1)
y_test = df_test['class']
```

FIGURE 3.5 – Séparation des variables explicatives et de l'étiquette

3.5 Implémentation des algorithmes de Machine Learning

Avant de passer à l'évaluation concrète des modèles d'apprentissage automatique, il est important de poser quelques bases essentielles. Dans cette phase, nous allons entraîner différents algorithmes sur notre jeu de données prétraité, puis mesurer leur performance sur des données de test.

Pour cela, nous utilisons plusieurs outils d'évaluation standards du *Machine Learning*. Ces indicateurs seront précieux pour comparer les différents algorithmes implémentés et sélectionner celui offrant les meilleures performances en matière de détection d'intrusions.

Pour chacun des trois modèles, le seuil de décision n'est pas fixé à 0,5 : il est choisi à partir de la courbe précision-rappel de façon à maximiser le score F1 de la classe `attack`. C'est avec ce seuil que sont calculés les rapports de classification et les matrices de confusion présentés plus loin.

3.5.1 Rapport de classification

Le **rapport de classification** est un tableau récapitulatif qui présente les principales métriques permettant d'évaluer la performance d'un modèle de machine learning pour une tâche de classification. Il fournit notamment :

- **La précision (Precision)** : proportion des prédictions positives correctes parmi l'ensemble des prédictions positives.
- **Le rappel (Recall)** : proportion des exemples positifs correctement identifiés par le modèle.
- **Le score F1** : moyenne harmonique entre la précision et le rappel, utile en cas de classes déséquilibrées.
- **L'exactitude (Accuracy)** : pourcentage global de bonnes prédictions.

Ce rapport permet non seulement de mesurer la qualité globale du modèle, mais aussi d'évaluer sa capacité à détecter avec précision chaque classe, ce qui est essentiel dans les contextes où certaines attaques sont rares mais critiques.

3.5.2 Matrice de confusion

La **matrice de confusion** permet de représenter de manière précise les performances d'un modèle de classification en comparant ses prédictions aux valeurs réelles. Elle organise les résultats en quatre catégories :

— **True Positive (TP)** : le modèle prédit une classe positive et celle-ci est effectivement correcte (ex. : une personne malade correctement identifiée comme malade).
— **True Negative (TN)** : le modèle indique une classe négative et cela correspond à la réalité (ex. : une personne en bonne santé prédite comme telle).
— **False Positive (FP)** : le modèle prévoit une classe positive alors que l'exemple est en réalité négatif (ex. : une personne saine classée à tort comme malade).
— **False Negative (FN)** : le modèle prévoit une classe négative alors qu'elle est en réalité positive (ex. : une personne malade considérée à tort comme en bonne santé).

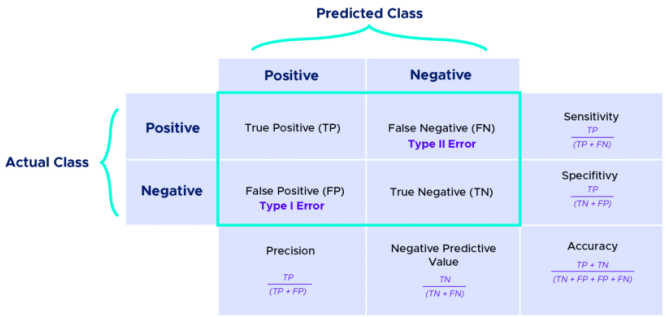


FIGURE 3.6 – Matrice de confusion [17]

3.5.3 Courbe ROC

La **courbe ROC** est un outil graphique qui permet d'évaluer la performance d'un modèle binaire selon différents seuils de décision. Elle trace la relation entre la capacité du modèle à bien détecter les éléments positifs (*taux de vrais positifs*) et sa tendance à générer de fausses alertes (*taux de faux positifs*).

Plus la courbe est proche de l'angle supérieur gauche, meilleure est la séparation entre les classes. L'aire sous la courbe (**AUC**) fournit une mesure globale : plus elle est élevée, plus le modèle est considéré comme performant.

3.5.4 Courbe d'apprentissage

La **courbe d'apprentissage** est un graphique qui illustre l'évolution des performances d'un modèle en fonction de la taille de l'échantillon d'entraînement. Elle permet de comparer le score obtenu sur les données d'apprentissage avec celui obtenu sur les données de validation. C'est un outil précieux pour évaluer la capacité du modèle à généraliser et pour ajuster les paramètres ou la quantité de données utilisées. Dans notre cas, elle est calculée par validation croisée à 5 plis sur le jeu d'entraînement, avec l'exactitude comme score.

3.5.5 SVM

L'algorithme **Support Vector Machine (SVM)** a été implémenté en utilisant un *pipeline* combinant la standardisation des données via `StandardScaler` et l'application du

classificateur SVM avec un noyau radial (RBF), de paramètres $C = 1{,}0$ et $\gamma =$ `'scale'`. Cette méthode vise à trouver une frontière de séparation optimale entre les classes, en maximisant la marge entre les données normales et les attaques. La standardisation est ici indispensable, car le SVM repose sur des calculs de distance : sans elle, les attributs de plus grande échelle (comme le nombre d'octets échangés) prendraient le dessus sur les autres. L'option `probability=True` est activée afin d'obtenir des probabilités et de pouvoir appliquer un seuil de décision.

```python
# Pipeline SVM avec standardisation
svm_pipeline = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)  # pour utiliser predict_proba()
)

print(" Entraînement du modèle SVM...")
svm_pipeline.fit(X_train, y_train)
```

FIGURE 3.7 – Standardisation des données dans le cadre de l'algorithme SVM

Une fois le modèle entraîné, des prédictions ont été réalisées sur les données de test, avec un seuil de décision de 0,009 (F1 = 0,883 pour la classe `attack`). La qualité des résultats a été analysée à travers plusieurs outils, notamment :

— **Le rapport de classification :**

```
Rapport de classification - SVM :
              precision    recall  f1-score   support

      attack       0.91      0.86      0.88     12833
      normal       0.82      0.89      0.86      9711

    accuracy                           0.87     22544
   macro avg       0.87      0.87      0.87     22544
weighted avg       0.87      0.87      0.87     22544

Accuracy : 0.8712739531582683
```

FIGURE 3.8 – Tableau de rapport de classification de l'algorithme SVM

Pour la classe `attack`, la précision est de 0,91 et le rappel de 0,86 : la grande majorité des alertes émises sont justifiées, mais environ une attaque sur sept n'est pas détectée. Pour la classe `normal`, la précision est de 0,82 et le rappel de 0,89, ce qui signifie que près de 9 connexions normales sur 10 sont correctement reconnues, mais que certaines attaques passent pour du trafic normal.

Le score F1 est de 0,88 pour `attack` et de 0,86 pour `normal`. L'accuracy globale est de 0,87, soit **87 % de prédictions correctes** sur les 22 544 connexions du jeu de test. Les moyennes macro et pondérée sont toutes deux à 0,87.

— **La matrice de confusion :**

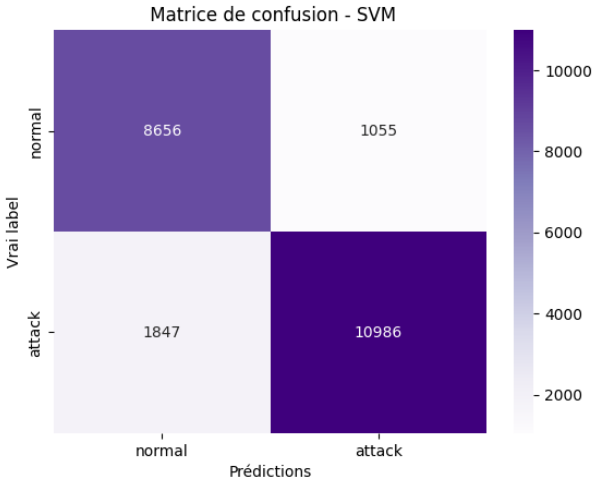


FIGURE 3.9 – Matrice de confusion de l'algorithme SVM

Le modèle a correctement identifié 8 656 connexions normales (True Negative) et 10 986 attaques (True Positive). Il a en revanche classé à tort 1 847 attaques comme normales (False Negative) et 1 055 connexions normales comme des attaques (False Positive).

Le SVM commet donc à la fois plus de faux négatifs et plus de faux positifs que les deux autres modèles étudiés plus loin. Les attaques non détectées sont les plus gênantes dans un contexte de détection d'intrusion, puisqu'elles passent inaperçues.

— **La courbe ROC :**

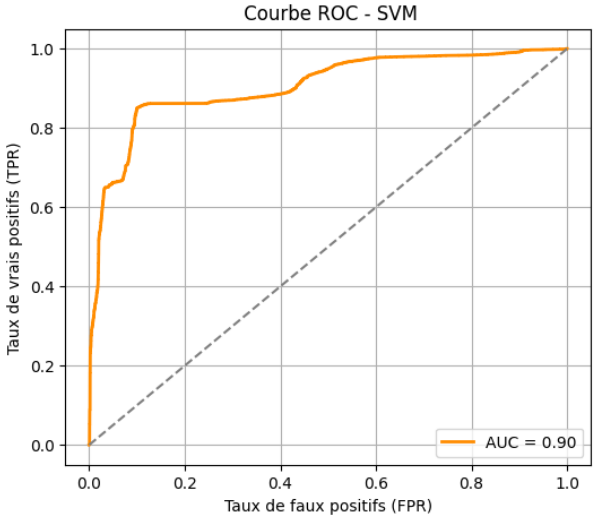


FIGURE 3.10 – Courbe ROC de l'algorithme SVM

La courbe s'éloigne nettement de la diagonale, ce qui montre que le modèle sépare correctement les deux classes. L'AUC est de 0,90, un bon score, mais inférieur à celui des deux autres modèles (0,97).

— **La courbe d'apprentissage :**

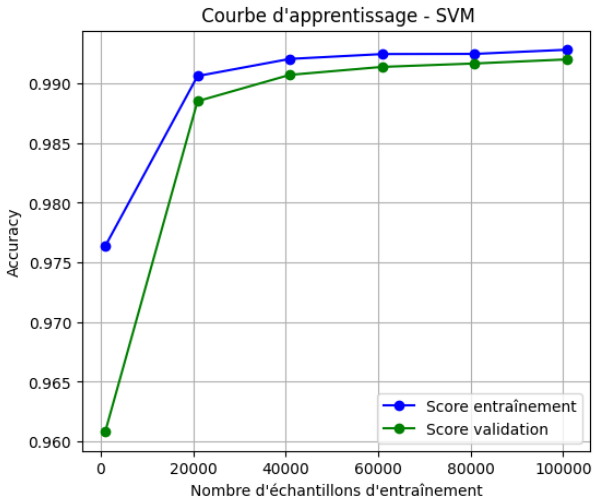


FIGURE 3.11 – Courbe d'apprentissage de l'algorithme SVM

On observe que le score d'entraînement est élevé dès les premières tailles d'échantillon (environ 0,976) et se stabilise autour de 0,993. Le score de validation progresse rapidement, de 0,961 à environ 0,992, et reste très proche de celui de l'entraînement. Cette faible distance entre les deux courbes indique que le modèle ne souffre pas d'un surapprentissage marqué sur le jeu d'entraînement. Il faut toutefois rappeler que ces scores sont obtenus par validation croisée sur KDDTrain+ : ils sont plus élevés que l'exactitude mesurée sur KDDTest+, dont les connexions diffèrent davantage de celles de l'entraînement.

3.5.6 Random Forest

L'algorithme **Random Forest** repose sur un ensemble d'arbres de décision, chacun construit à partir de sous-échantillons aléatoires de données et de variables. La décision finale est obtenue en agrégeant les prédictions de tous les arbres, ce qui réduit la variance par rapport à un arbre isolé.

Le modèle a été configuré avec 500 arbres, une profondeur maximale de 15, un minimum de 10 échantillons pour diviser un nœud (`min_samples_split`) et de 4 échantillons par feuille (`min_samples_leaf`), et le critère `max_features='sqrt'`. Ces paramètres limitent la complexité des arbres et réduisent le risque de surapprentissage. Le seuil de décision retenu est de 0,018 (F1 = 0,935 pour la classe `attack`).

Après l'entraînement sur `X_train` et `y_train`, les performances ont été évaluées à l'aide des outils suivants :

— **Le rapport de classification :**

```
Rapport de classification - Random Forest :
              precision    recall  f1-score   support

      attack       0.89      0.99      0.93     12833
      normal       0.98      0.84      0.90      9711

    accuracy                           0.92     22544
   macro avg       0.93      0.91      0.92     22544
weighted avg       0.93      0.92      0.92     22544

Accuracy : 0.921930447125621
```

FIGURE 3.12 – Tableau de rapport de classification de l'algorithme Random Forest

Pour la classe `attack`, le modèle obtient un rappel de 0,99 pour une précision de 0,89 : presque toutes les attaques sont détectées, au prix de quelques fausses alertes. Pour la classe `normal`, la précision est très élevée (0,98), ce qui signifie que lorsque le modèle juge une connexion normale, il se trompe rarement, tandis que le rappel est de 0,84, donc une partie des connexions normales est classée à tort comme attaque.

Le score F1 est de 0,93 pour `attack` et de 0,90 pour `normal`. L'accuracy globale atteint 0,92 (exactement 92,19 %), et les moyennes macro sont de 0,93 pour la précision, 0,91 pour le rappel et 0,92 pour le F1.

— **La matrice de confusion :**

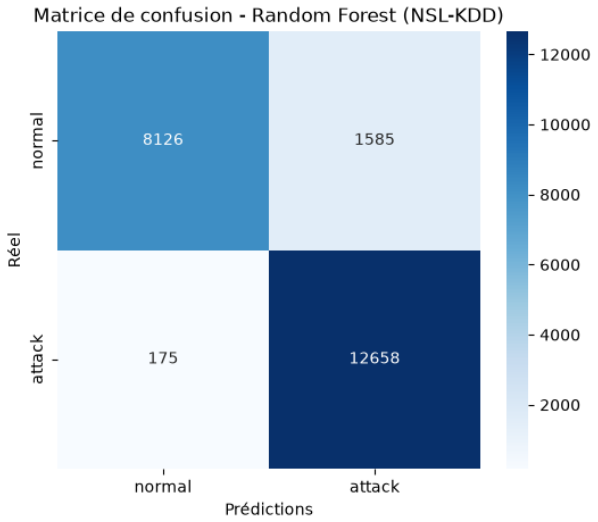


FIGURE 3.13 – Matrice de confusion de l'algorithme Random Forest

La matrice de confusion montre les résultats suivants :

— 8 126 connexions normales ont été correctement classées, contre 1 585 faux positifs.

— Pour les attaques, 12 658 ont été bien détectées et seulement 175 ont été classées à tort comme normales (faux négatifs).

Le nombre de faux négatifs est donc très faible, ce qui est le point le plus important pour un système de détection d'intrusion. En contrepartie, le modèle génère un nombre plus élevé de fausses alertes (1 585), conséquence du seuil bas qui privilégie la détection des attaques.

— **La courbe ROC :**

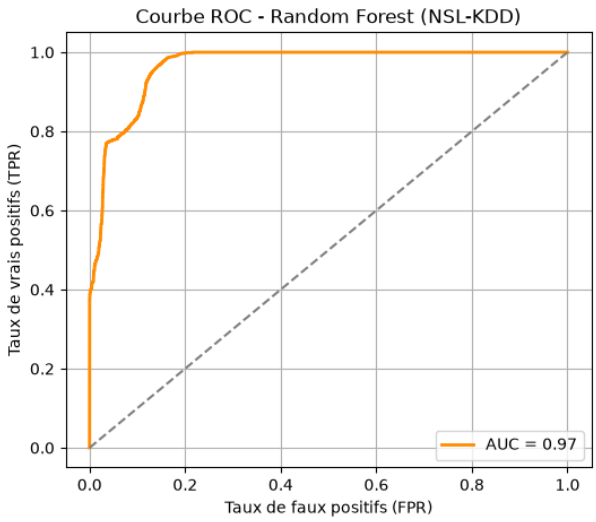


FIGURE 3.14 – Courbe ROC de l'algorithme Random Forest

La courbe ROC s'élève rapidement vers le coin supérieur gauche, ce qui reflète un taux élevé de vrais positifs pour un faible taux de faux positifs. L'AUC atteint 0,97 (0,9657), ce qui indique une très bonne capacité à distinguer les comportements normaux des intrusions, quel que soit le seuil de décision choisi.

— **La courbe d'apprentissage :**

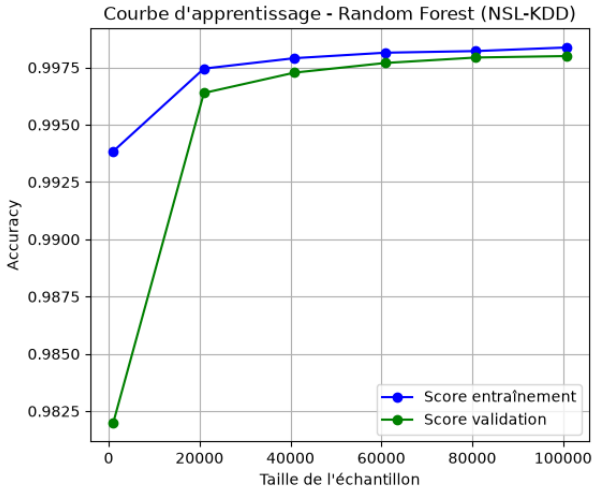


FIGURE 3.15 – Courbe d'apprentissage de l'algorithme Random Forest

La courbe d'apprentissage montre une bonne stabilité du modèle dès les premières tailles d'échantillon. Le score d'entraînement est très élevé (de 0,994 à environ 0,998) et le score de validation progresse rapidement, de 0,982 à environ 0,998. L'écart entre les deux courbes devient presque nul, ce qui indique que le modèle généralise bien sur les données d'entraînement et n'est pas en situation de surapprentissage marqué. Comme pour le SVM, ces valeurs proviennent d'une validation croisée sur KDDTrain+ et ne sont pas directement comparables à l'exactitude obtenue sur KDDTest+.

3.5.7 XGBoost

L'algorithme **XGBoost** est une implémentation optimisée du *gradient boosting* : les arbres de décision sont construits successivement, chaque nouvel arbre cherchant à corriger les erreurs commises par les précédents. Il est reconnu pour son efficacité dans le traitement des données structurées.

Le modèle a été configuré avec 500 estimateurs, une profondeur maximale de 8 et un taux d'apprentissage de 0,05, avec un échantillonnage de 80 % des observations (`subsample`) et des attributs (`colsample_bytree`) pour chaque arbre. Une régularisation L1 ($\alpha = 0{,}1$) et L2 ($\lambda = 1{,}0$) est également appliquée pour limiter le surapprentissage. Le seuil de décision retenu est très faible (de l'ordre de 7×10^{-5}), avec un F1 de 0,929 pour la classe `attack`.

Une fois entraîné, les prédictions ont été effectuées sur les données de test :

— **Le rapport de classification :**

```
Rapport de classification - XGBoost :
              precision    recall  f1-score   support

      attack       0.88      0.99      0.93     12833
      normal       0.98      0.82      0.89      9711

    accuracy                           0.91     22544
   macro avg       0.93      0.90      0.91     22544
weighted avg       0.92      0.91      0.91     22544

Accuracy : 0.914611426543648
```

FIGURE 3.16 – Tableau de rapport de classification de l'algorithme XGBoost

Ce rapport met en évidence un rappel de 0,99 pour la classe `attack`, ce qui signifie que presque toutes les attaques sont détectées, avec une précision de 0,88. À l'inverse, la classe `normal` présente une précision de 0,98 mais un rappel plus modeste (0,82), indiquant qu'une partie des connexions normales est confondue avec des attaques.

Le score F1 est de 0,93 pour `attack` et de 0,89 pour `normal`. Le score global d'accuracy atteint 0,91 (91,46 %), et les moyennes macro sont de 0,93 pour la précision, 0,90 pour le rappel et 0,91 pour le F1.

— **La matrice de confusion :**

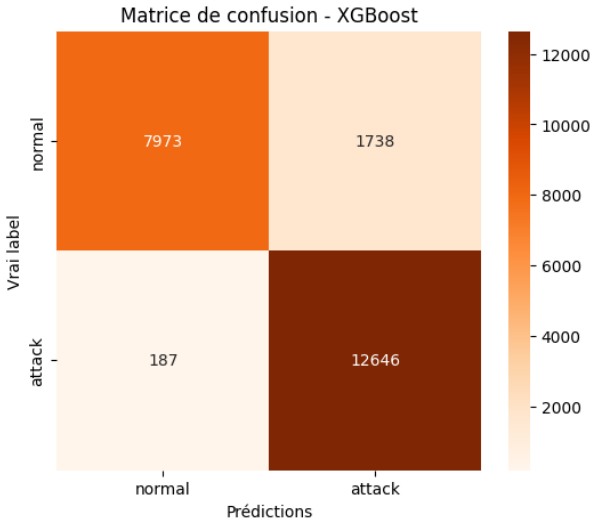


FIGURE 3.17 – Matrice de confusion de l'algorithme XGBoost

Le modèle a correctement identifié 7 973 connexions normales et 12 646 attaques. Il a classé à tort 187 attaques comme normales et 1 738 connexions normales comme des attaques. Le nombre de faux négatifs est très faible, comme pour le Random Forest, mais le nombre de fausses alertes est un peu plus élevé (1 738 contre 1 585).

— **La courbe ROC :**

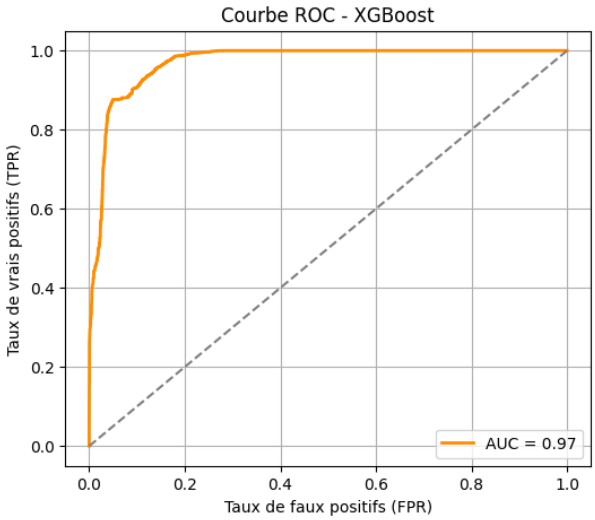


FIGURE 3.18 – Courbe ROC de l'algorithme XGBoost

La courbe ROC du modèle montre une très bonne capacité à distinguer les deux classes. Elle s'élève rapidement vers le coin supérieur gauche. L'AUC atteint 0,97 (0,9702), la valeur la plus élevée des trois modèles, ce qui traduit la fiabilité du modèle pour séparer les attaques des connexions normales, quel que soit le seuil de classification choisi.

— **La courbe d'apprentissage :**

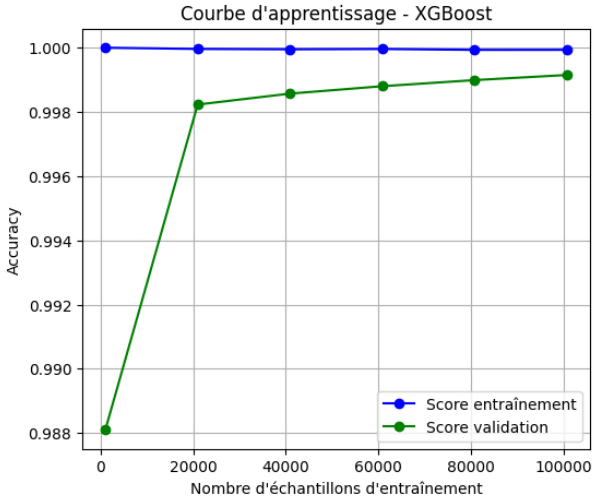


FIGURE 3.19 – Courbe d'apprentissage de l'algorithme XGBoost

Le score d'entraînement est pratiquement égal à 1,000 sur toutes les tailles d'échantillon, tandis que le score de validation passe de 0,988 à environ 0,999 lorsque la taille de l'échantillon augmente. L'écart entre les deux courbes reste très faible, et la performance se stabilise dès 20 000 échantillons environ : le modèle n'a donc pas besoin d'une quantité très importante de données supplémentaires pour atteindre son niveau de performance.

3.6 Analyse des résultats

Une fois les différents modèles de Machine Learning entraînés et évalués sur les mêmes données, il devient essentiel de procéder à une comparaison approfondie afin de déterminer lequel est le plus adapté à notre problématique.

Chaque algorithme possède ses propres mécanismes de traitement, ses avantages et ses limites, ce qui se reflète dans les résultats obtenus.

Dans cette section, nous analysons et confrontons les performances de SVM, Random Forest et XGBoost en nous appuyant sur plusieurs indicateurs : l'accuracy, le rapport de classification, la matrice de confusion et l'AUC.

L'évaluation des différents modèles d'apprentissage supervisé a permis de mettre en lumière les forces et les faiblesses de chaque approche dans le cadre de la détection d'intrusion. Le SVM obtient des résultats corrects (accuracy de 87,13 %, AUC de 0,90) mais reste nettement en retrait : il laisse passer 1 847 attaques et génère 1 055 fausses alertes. Random Forest et XGBoost se démarquent tous deux par un rappel de 0,99 sur les attaques (respectivement 175 et 187 attaques non détectées) et par une AUC de 0,97. Random Forest obtient la meilleure accuracy (92,19 % contre 91,46 % pour XGBoost) et un peu moins de fausses alertes (1 585 contre 1 738), tandis que XGBoost présente la meilleure AUC (0,9702 contre 0,9657). Les deux modèles étant très proches, c'est le Random Forest qui a été retenu pour l'interface de prédiction décrite au chapitre suivant, en raison de son accuracy et de son nombre de fausses alertes légèrement meilleurs.

Voici un tableau qui illustre des performances de chacun des algorithmes :

Modèle	**Points forts**	**Limites principales**	**Verdict général**
SVM	Bonne précision sur les attaques (0,91), AUC de 0,90	Accuracy la plus faible (0,87), 1 847 attaques non détectées	Utilisable, mais en retrait par rapport aux deux autres modèles
Random Forest	Meilleure accuracy (0,92), rappel de 0,99 sur les attaques, AUC de 0,97	Rappel de 0,84 sur la classe normal, 1 585 fausses alertes	L'un des deux modèles les plus performants, retenu pour l'interface
XGBoost	Rappel de 0,99 sur les attaques, meilleure AUC (0,9702)	Rappel de 0,82 sur la classe normal, 1 738 fausses alertes	L'un des deux modèles les plus performants, très proche du Random Forest

TABLE 3.1 – Comparaison des modèles de classification pour la détection d'intrusion

3.7 Conclusion

L'évaluation comparative des différents algorithmes testés dans le cadre de notre système de détection d'intrusion a permis de mettre en évidence les forces et les faiblesses de chacun. Parmi eux, **Random Forest et XGBoost se sont imposés comme les modèles les plus performants**, avec une AUC de 0,97 et un rappel de 0,99 sur les attaques, loin devant le SVM. Le Random Forest présente la meilleure accuracy (92,19 %) et le XGBoost la meilleure AUC.

Les résultats des deux modèles étant très proches, le choix final s'est porté sur le **Random Forest**, qui offre la meilleure exactitude globale et un peu moins de fausses alertes, tout en détectant presque toutes les intrusions.

Maintenant que le modèle a été identifié et validé, la prochaine étape consistera à l'intégrer dans une solution fonctionnelle, à travers une interface utilisateur interactive. Cette interface permettra d'exploiter le modèle de manière intuitive, en facilitant l'analyse de nouveaux flux de données réseau et en affichant les résultats de détection de façon claire et exploitable.

Ce déploiement pratique vise à transformer notre modèle théorique en un outil concret, utile pour la surveillance et la sécurité des systèmes informatiques.

Chapitre 4

Intégration d'une interface utilisateur de prédiction

4.1 Introduction

Une fois les modèles entraînés, testés et comparés, l'étape suivante consiste à rendre le système fonctionnel et accessible à travers une interface conviviale. L'objectif n'est plus seulement de démontrer la performance du modèle dans un cadre expérimental, mais de le rendre exploitable par un utilisateur final de manière simple, rapide et autonome.

Dans cette optique, une interface utilisateur a été développée afin de faciliter l'interaction avec le système de détection d'intrusion. Cette interface permet d'exploiter les capacités du modèle dans un environnement plus intuitif, sans nécessiter de compétences techniques approfondies. Elle représente ainsi le lien entre la phase de conception du modèle et son application concrète, ouvrant la voie à une intégration dans un cadre réel, où la détection d'intrusion peut être effectuée sur de nouvelles données de manière interactive.

4.2 Structure générale

Le code de l'interface a été développé avec **Streamlit**, une bibliothèque Python permettant de créer facilement des applications web interactives. La première section configure l'apparence générale de la page, en définissant un titre, un sous-titre explicatif et une mise en page centrée pour une meilleure lisibilité.

Ensuite, la fonction `load_artifacts()` est utilisée pour charger les trois éléments enregistrés à la fin de l'entraînement du modèle **Random Forest**, retenu à l'issue de la comparaison : le modèle lui-même (`rf_model.pkl`), la liste des 122 colonnes attendues (`rf_columns.pkl`) et le seuil de décision (`rf_threshold.pkl`). Ces fichiers sont lus avec `joblib` à partir du dossier du script, et la fonction est optimisée grâce à un mécanisme de cache (`@st.cache_resource`) afin d'accélérer l'exécution lors des utilisations répétées. Si l'un des fichiers est introuvable, un message d'erreur explicite est affiché et l'application s'arrête.

```python
from pathlib import Path
BASE = Path(__file__).parent

@st.cache_resource
def load_artifacts():
    model = joblib.load(BASE / "rf_model.pkl")
    columns = joblib.load(BASE / "rf_columns.pkl")
    threshold = joblib.load(BASE / "rf_threshold.pkl")
    return model, columns, threshold

try:
    rf_model, expected_columns, default_threshold = load_artifacts()
    st.sidebar.success("Modèle chargé avec succès ")
except FileNotFoundError:
    st.error(
        "Fichiers du modèle introuvables. Assure-toi que `rf_model.pkl`, "
        "`rf_columns.pkl` et `rf_threshold.pkl` sont dans le même dossier "
        "que ce script (générés depuis le notebook avec `joblib.dump`)."
    )
    st.stop()
```

FIGURE 4.1 – Configuration de l'interface et chargement du modèle

L'interface propose à l'utilisateur de téléverser un fichier CSV contenant les données à analyser. Ce fichier doit avoir déjà subi le prétraitement décrit au chapitre précédent : il contient les 122 colonnes obtenues après l'encodage one-hot, au même format que `X_train` et `X_test` dans le notebook. Le fichier est lu et automatiquement vérifié par rapport aux colonnes attendues par le modèle. Si des colonnes manquent, un message d'avertissement est affiché pour guider l'utilisateur ; si le fichier contient des colonnes supplémentaires, elles sont simplement ignorées et l'utilisateur en est informé.

Une colonne `class` facultative peut également être présente : lorsque c'est le cas, elle est utilisée pour comparer les prédictions aux étiquettes réelles. Une barre latérale permet en outre de modifier le seuil de décision à l'aide d'un curseur, la valeur par défaut étant celle enregistrée lors de l'entraînement.

```python
uploaded_file = st.file_uploader(
    "Choisis un fichier CSV déjà prétraité (122 colonnes)",
    type=["csv"]
)

    df_raw = pd.read_csv(uploaded_file)
except Exception as e:
    st.error(f"Erreur lors de la lecture du fichier CSV : {e}")
    st.stop()
```

FIGURE 4.2 – Téléverser un fichier CSV

Voici un screenshot de notre interface qui illustre la page d'accueil et la fonctionnalité de téléversement de fichier, comme décrit précédemment. Cette étape permet à l'utilisateur d'importer un fichier `.csv` contenant les données à analyser, qui seront ensuite automatiquement traitées par le modèle.

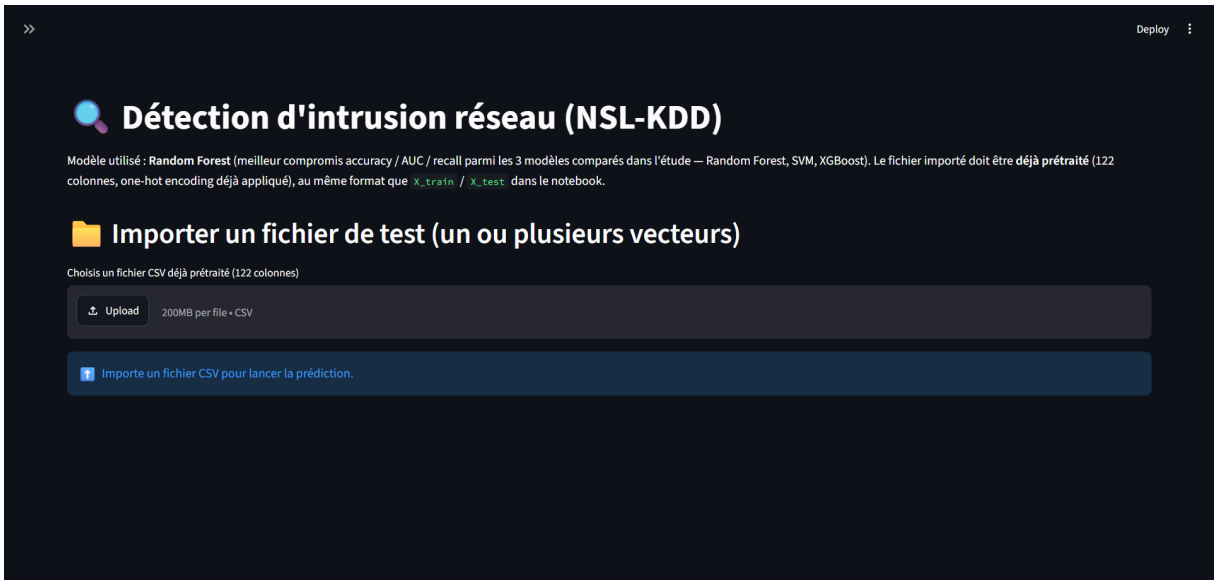


FIGURE 4.3 – Screenshot de l'accueil de l'interface

Si le fichier est valide, l'application passe à la phase de prédiction. Les colonnes du fichier sont d'abord réalignées sur l'ordre exact utilisé lors de l'entraînement, puis le modèle calcule pour chaque connexion une probabilité d'attaque. Cette probabilité est comparée au seuil de décision : au-dessus du seuil, la connexion est classée `attack`, sinon elle est classée `normal`. Le résultat est ajouté au tableau sous forme de deux nouvelles colonnes, `probabilite_attack` et `prediction`. Enfin, les erreurs potentielles sont gérées via des blocs `try-except`, assurant une expérience fluide et sans plantage.

```
# Réalignement strict sur l'ordre exact des colonnes utilisées à l'entraînement
df_encoded = df_features.reindex(columns=expected_columns, fill_value=0)

attack_index = list(rf_model.classes_).index("attack")
y_score = rf_model.predict_proba(df_encoded)[:, attack_index]
y_pred = np.where(y_score >= threshold, "attack", "normal")

results_df = df_raw.copy()
results_df["probabilite_attack"] = y_score
results_df["prediction"] = y_pred
```

FIGURE 4.4 – Phase de prédiction

Voici d'autres captures d'écran illustrant l'utilisation de l'interface avec un fichier `deux_lignes.csv` contenant deux connexions. Après l'import, un aperçu des données chargées est affiché, avec le nombre de vecteurs et de colonnes détectés.



FIGURE 4.5 – Résultat de l'import d'un fichier CSV et aperçu des données

L'interface affiche ensuite un résumé global des prédictions : le nombre de vecteurs analysés, le nombre d'attaques détectées, le nombre de connexions normales et la probabilité moyenne d'attaque. Dans cet exemple, une connexion sur deux est classée comme attaque et la probabilité moyenne est de 0,287. Le tableau détaillé colore chaque ligne selon la prédiction (rouge pour une attaque, vert pour une connexion normale), et un bouton permet de télécharger l'ensemble des résultats au format CSV.

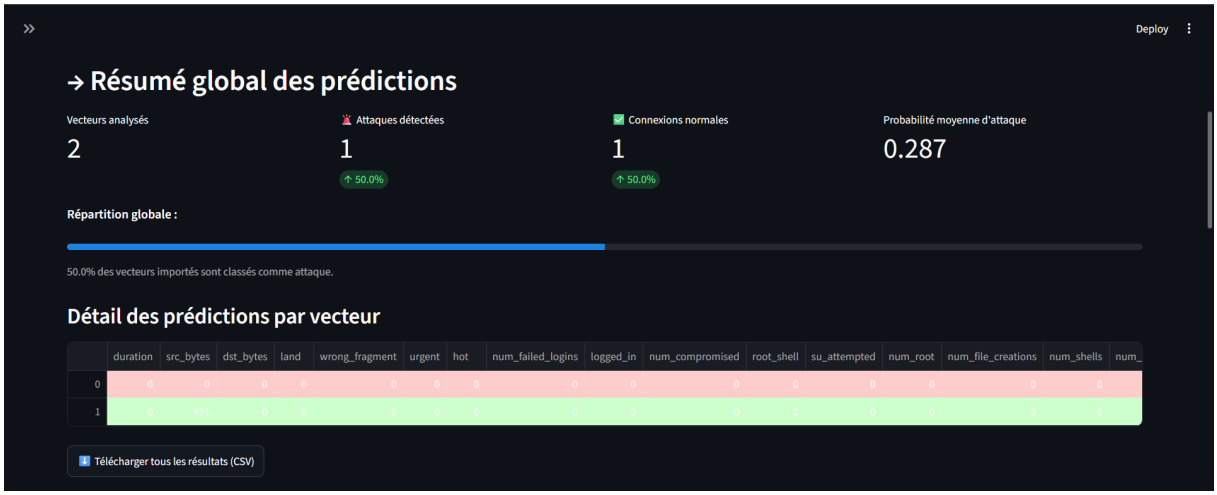


FIGURE 4.6 – Résumé global et tableau détaillé des prédictions

Lorsque le fichier contient plusieurs lignes, deux graphiques sont aussi affichés : le premier illustre le nombre d'occurrences de chaque classe (`attack` ou `normal`) parmi les prédictions effectuées, et le second représente la distribution des probabilités d'attaque, avec le seuil de décision (0,0180) matérialisé par une ligne verticale en pointillés.

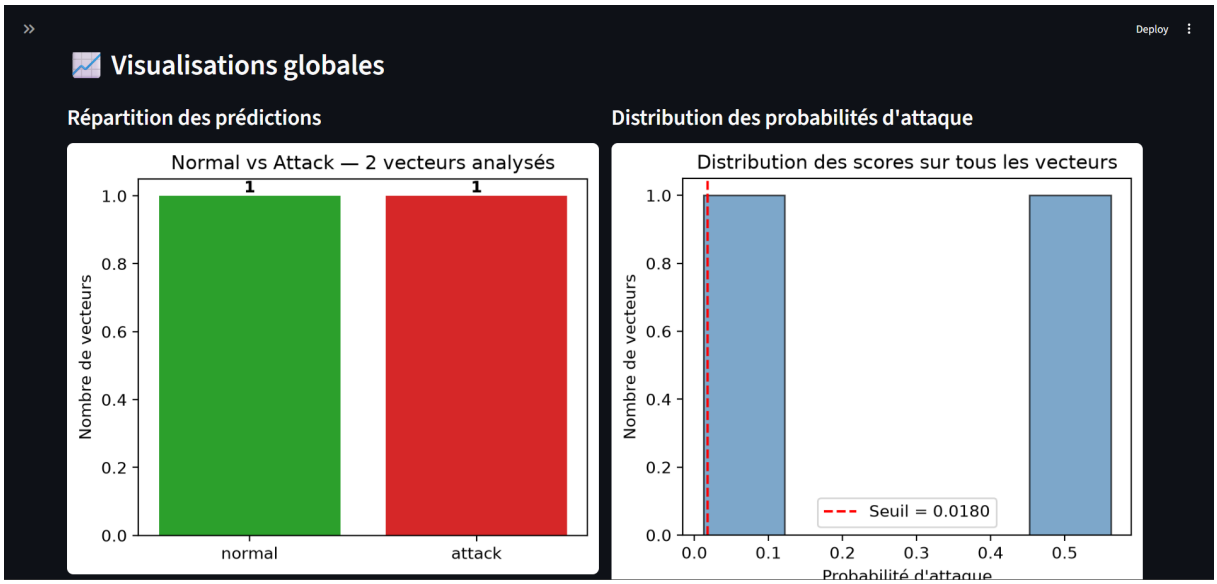


FIGURE 4.7 – Graphiques de répartition des prédictions et de distribution des probabilités d'attaque

Un troisième graphique, en secteurs, présente la répartition globale entre les connexions classées normales et celles classées attaques.

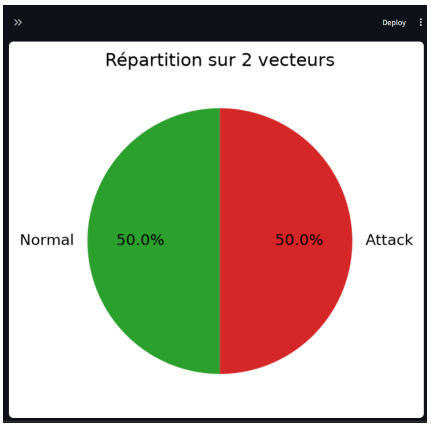


FIGURE 4.8 – Répartition globale des prédictions sous forme de graphique en secteurs

4.3 Conclusion

Le développement de cette interface marque l'aboutissement pratique de notre travail. En transformant un modèle de machine learning en un outil interactif et accessible, elle permet une mise en application concrète du système de détection d'intrusion.

Grâce à une interface simple et intuitive, l'utilisateur peut tester de nouvelles données en quelques clics, sans avoir besoin de connaissances techniques avancées. Cette étape vient compléter la chaîne de traitement construite au fil du projet, en facilitant l'exploitation du modèle dans un environnement réel.

Elle constitue également une base solide pour un déploiement futur, que ce soit en milieu professionnel ou éducatif.

Conclusion générale

À travers ce travail, nous avons exploré l'un des domaines les plus critiques de la cybersécurité : la détection des intrusions. En combinant des approches théoriques solides avec une mise en œuvre pratique basée sur le machine learning, notre objectif a été de concevoir un système capable d'analyser le trafic réseau et d'identifier les comportements malveillants avec efficacité.

La première partie du rapport a permis d'établir un cadre conceptuel clair en présentant les fondements des systèmes de détection d'intrusion (IDS) ainsi que les principes de l'apprentissage automatique appliqués à la sécurité informatique. Cette base nous a ensuite permis d'entrer dans la phase pratique, où nous avons travaillé avec le dataset NSL-KDD, appliqué plusieurs algorithmes de classification (SVM, Random Forest et XGBoost), comparé leurs performances, et identifié Random Forest et XGBoost comme les modèles les plus performants dans notre contexte, avec une AUC de 0,97 chacun et un rappel de 0,99 sur les attaques. Random Forest présente la meilleure exactitude (92,19 %) et a été retenu pour la suite du projet.

L'intégration du modèle final dans une interface utilisateur simple et fonctionnelle a constitué une étape importante vers une solution exploitable. Malgré certaines limites matérielles et temporelles, ce projet nous a offert une expérience enrichissante, tant sur le plan technique que méthodologique.

En conclusion, ce travail pose les fondations d'un système de détection intelligent et évolutif. Il ouvre également la voie à plusieurs pistes d'amélioration, notamment l'utilisation de jeux de données plus récents comme CICIDS2017, l'approche multi-classes pour identifier le type d'attaque, ou encore le déploiement du système dans un environnement réel pour une détection en temps réel. Ce projet marque ainsi une première étape vers des solutions de sécurité plus autonomes, robustes et adaptées aux menaces actuelles.

Annexe

Implémentation

Introduction

Ce chapitre a pour but de décrire les outils de développement, les bibliothèques Python et l'environnement matériel qui ont permis la mise en œuvre concrète de notre système de détection d'intrusion. Chaque élément a été choisi pour sa pertinence, sa simplicité d'intégration et son efficacité dans le traitement de données complexes.

Nous reviendrons sur les principales technologies utilisées, qu'il s'agisse de l'environnement de programmation. Enfin, une brève description du matériel utilisé permettra de mieux comprendre les limites techniques rencontrées lors de l'entraînement et de l'évaluation des modèles.

Outils de développement

Environnement de développement

Visual Studio Code (VS Code) est un éditeur de code source ouvert et polyvalent développé par Microsoft. Il se distingue par sa compatibilité avec de nombreux langages de programmation, associée à une vaste bibliothèque d'extensions, permettant de l'adapter facilement aux besoins spécifiques des développeurs.

L'éditeur propose également une interface personnalisable et une assistance intelligente au code, incluant l'auto-complétion, la refactorisation et les suggestions contextuelles. De plus, VS Code intègre un terminal intégré et une gestion efficace des espaces de travail, optimisant ainsi le flux de travail des développeurs.



FIGURE 9 – Logo de Visual Studio Code

Langage de programmation

Python est un langage de programmation polyvalent, réputé pour sa simplicité et sa lisibilité. Il dispose d'une syntaxe claire et structurée, d'une vaste collection de bibliothèques standard et de modules tiers, facilitant le développement dans des domaines variés. De plus, sa compatibilité avec des frameworks populaires le rend idéal pour des projets allant du traitement de données à l'intelligence artificielle.

En outre, Python est un langage interprété, ce qui favorise une phase de développement rapide et un débogage efficace.



FIGURE 10 – Logo de Python

Extensions

Jupyter est un environnement interactif basé sur des notebooks, permettant d'exécuter du code Python tout en intégrant des visualisations, des graphiques et des annotations. Idéal pour la data science et le machine learning, il facilite l'exploration des données, le prototypage de modèles et la documentation des analyses dans un format reproductible.



FIGURE 11 – Logo de Jupyter

Bibliothèques

Une bibliothèque logicielle, également appelée librairie ou bibliothèque de programmes, est un ensemble de fonctions utilitaires organisées et mises à disposition pour être réutilisées sans nécessiter une réécriture. Ces fonctions sont regroupées selon des domaines spécifiques tels que les mathématiques, le graphisme ou les algorithmes de tri [18].

Dans ce qui suit, nous allons définir toutes les bibliothèques utilisées dans notre implémentation :

Bibliothèques du prétraitement et manipulation des données

— **Pandas** : bibliothèque open-source dédiée à la manipulation et à l'analyse des données. Elle propose des structures comme les DataFrames et les Series, facilitant le traitement de données tabulaires étiquetées [19]. Elle fournit aussi la fonction `get_dummies`, utilisée pour l'encodage one-hot.

— **NumPy** : abréviation de “Numerical Python”, elle permet le calcul scientifique à l’aide de tableaux multidimensionnels et de fonctions mathématiques performantes [19].

— **Scikit-learn** :

  — **StandardScaler** : standardise les données (moyenne = 0, écart-type = 1), indispensable pour les modèles sensibles à l’échelle des données, comme les SVM [20].

Bibliothèques de modélisation et Machine Learning

— **Scikit-learn** :

  — **RandomForestClassifier** : algorithme d’apprentissage automatique qui entraîne plusieurs arbres de décision sur des sous-échantillons du jeu de données et combine leurs prédictions pour améliorer la précision et réduire le surapprentissage. Chaque arbre utilise la meilleure division possible des données [21].

  — **SVC** : cette classe implémente les Support Vector Machines, permettant de séparer des classes en maximisant la marge entre elles. Elle accepte différents noyaux (linéaire, RBF...) pour s’adapter à la nature des données [21].

  — **make_pipeline** : permet de construire un pipeline enchaînant plusieurs étapes (prétraitement, transformation, classification). Elle structure le code et facilite l’optimisation des hyperparamètres [22].

— **XGBoost** :

  — **XGBClassifier** : algorithme de Gradient Boosting optimisé pour la rapidité et la performance. Il gère les valeurs manquantes, la régularisation et le surapprentissage, ce qui le rend idéal pour les grands ensembles de données [23].

Bibliothèques d’évaluation, de visualisation, et d’interface

— **Bibliothèques de l’évaluation des modèles** :

  — **scikit-learn** :

    — `classification_report`, `accuracy_score`, `confusion_matrix` : outils d’évaluation et mesure des performances des modèles.

    — `roc_curve`, `auc` : permettent l’analyse des performances à l’aide de la courbe ROC et de l’aire sous cette courbe.

    — `precision_recall_curve` : permet de calculer la courbe précision-rappel utilisée pour choisir le seuil de décision.

    — `learning_curve` : pour analyser l’apprentissage et la généralisation du modèle.

— **Bibliothèques de visualisation graphique** :

  — **matplotlib.pyplot** : module de Matplotlib qui permet de créer des figures, des tracés, et de personnaliser l’affichage de manière similaire à MATLAB [24].

  — **seaborn** : bibliothèque construite sur Matplotlib, simplifie la création de visualisations statistiques élégantes [25].

— **Bibliothèque de sérialisation** :

— **joblib** : permet de sauvegarder et de recharger des objets Python volumineux, comme les modèles de machine learning. Elle repose sur le même principe que **pickle** (sérialisation et désérialisation d'objets Python [26]), mais elle est mieux adaptée aux objets contenant de grands tableaux numériques.

— **Bibliothèque d'interface utilisateur** :

— **streamlit** : transforme des scripts Python en applications web interactives, idéale pour la mise en production rapide de projets de data science [27].

— **Bibliothèques système et alertes** :

— **os** : permet l'interaction avec le système d'exploitation (fichiers, dossiers, variables d'environnement, etc.) [28].

— **warnings** : permet d'émettre des avertissements contrôlés sans interrompre l'exécution du programme [29].

Performance Evaluation

Tous les tests ont été effectués à l'aide d'un ordinateur portable avec les détails de configuration matérielle et logicielle indiqués dans le tableau ci-dessous :

Caractéristique	**Détail**
Fabricant du système	LENOVO
Type de système	Système d'exploitation 64 bits
Système opérateur	Windows 11 Professionnel
Processeur	AMD Ryzen 5 5500U avec Radeon Graphics (12 CPUs), ∼2.1GHz
Modèle du système	21A4
BIOS	GGCN29WW (V1.15)
Mémoire	12288MB RAM
Fichier de pagination	15631 Mo utilisés, 7231 Mo disponibles
Version DirectX	DirectX 12
Signature WHQL	Rechercher les signatures numériques WHQL

Table 1 – La configuration matérielle et logicielle

Conclusion

L'ensemble des outils et bibliothèques utilisés a constitué une base solide pour le développement de notre projet. Malgré certaines limites matérielles, nous avons pu mettre en place une solution fonctionnelle et exploitable, en tirant parti des ressources disponibles de manière optimale. Ces choix techniques ont joué un rôle déterminant dans la réussite de l'implémentation du système.

Perspectives et limites

Introduction

Bien que notre projet ait permis de concevoir et d'évaluer un système efficace de détection d'intrusion basé sur des algorithmes de machine learning, certaines pistes d'amélioration et d'approfondissement ont émergé au fil du travail. Ce chapitre présente les différentes orientations que nous avons envisagées pour enrichir notre approche et repousser les limites rencontrées.

Dans cette optique, nous avons exploré l'utilisation du dataset CICIDS2017, reconnu pour sa richesse et sa représentativité des attaques actuelles. De plus, nous avons tenté une approche multi-classes avec NSL-KDD afin d'identifier précisément le type d'attaque, et non simplement distinguer entre trafic normal ou malveillant. Toutefois, des contraintes techniques et matérielles, ainsi que le temps limité dont nous disposions, ont freiné la mise en œuvre complète de ces alternatives.

Cette section met ainsi en lumière les perspectives d'évolution de notre projet et les axes à explorer dans de futurs travaux, en vue de construire un système encore plus complet et performant.

Approche multi-classes avec NSL-KDD

Dans le but d'enrichir notre système de détection d'intrusion, nous avons envisagé de dépasser la simple classification binaire « normal » ou « attaque », en explorant une approche plus granulaire capable d'identifier précisément le type d'attaque détecté. Le dataset NSL-KDD contient en effet des étiquettes représentant différentes catégories d'intrusions telles que DoS, Probe, R2L et U2R, qui offrent un potentiel d'analyse bien plus détaillé.

Une telle classification aurait permis de fournir des informations plus spécifiques aux administrateurs système, facilitant ainsi des réponses adaptées à chaque menace. Toutefois, cette démarche s'est heurtée à plusieurs obstacles techniques : certaines classes comme U2R ou R2L sont sous-représentées, ce qui complique l'apprentissage et engendre un fort déséquilibre dans les prédictions.

De plus, cette approche multi-classes aurait nécessité des capacités de calcul plus importantes et un temps d'entraînement plus long, ce qui dépassait les moyens matériels et temporels à notre disposition dans le cadre de ce projet.

Exploration du dataset CICIDS2017

Dans l'optique de renforcer la pertinence et la modernité de notre projet, nous avons également porté notre attention sur le dataset CICIDS2017, une base de données plus récente, conçue pour représenter avec fidélité le trafic réseau réel, incluant une large variété d'attaques contemporaines. CICIDS2017 offre un environnement de simulation plus proche des infrastructures actuelles, avec des attributs plus riches et une granularité plus fine.

L'idée était de comparer les performances des modèles sur deux contextes différents, c'est-à-dire sur les deux bases de données, afin d'évaluer leur capacité de généralisation. Cependant, la taille conséquente du dataset et la complexité de son format ont constitué un frein. Le volume important de données requiert des ressources de traitement et de stockage avancées, que nous ne pouvions mobiliser dans le cadre de ce travail.

Afin d'illustrer concrètement les propos précédents, nous présentons ci-dessous un exemple représentatif des résultats obtenus lors de notre expérimentation.

Nous donnons ici une interprétation des résultats trouvés pour le modèle XGBoost, l'un des deux modèles les plus performants sur le dataset NSL-KDD, afin d'évaluer son comportement sur la base de données CICIDS2017. Les résultats obtenus sont remarquablement élevés :

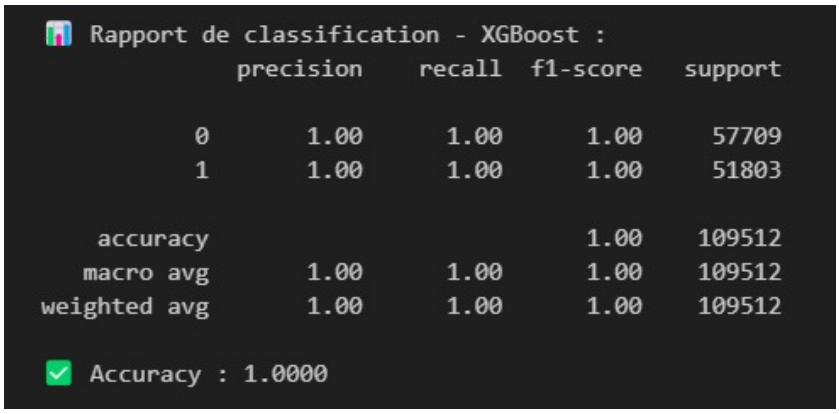


FIGURE 12 – Rapport de classification de l'algorithme XGBoost sur CICIDS2017

Analyse des résultats du modèle XGBoost sur CICIDS2017

Le rapport de classification montre une précision, un rappel et un score F1 parfaits (1.00) pour les deux classes (normal et attack), avec une accuracy de 100 % sur un ensemble de plus de 100 000 connexions.

La matrice de confusion vient confirmer ce constat, avec seulement une seule erreur de classification sur plus de 109 000 échantillons, ce qui indique une capacité exceptionnelle du modèle à distinguer les comportements malveillants.

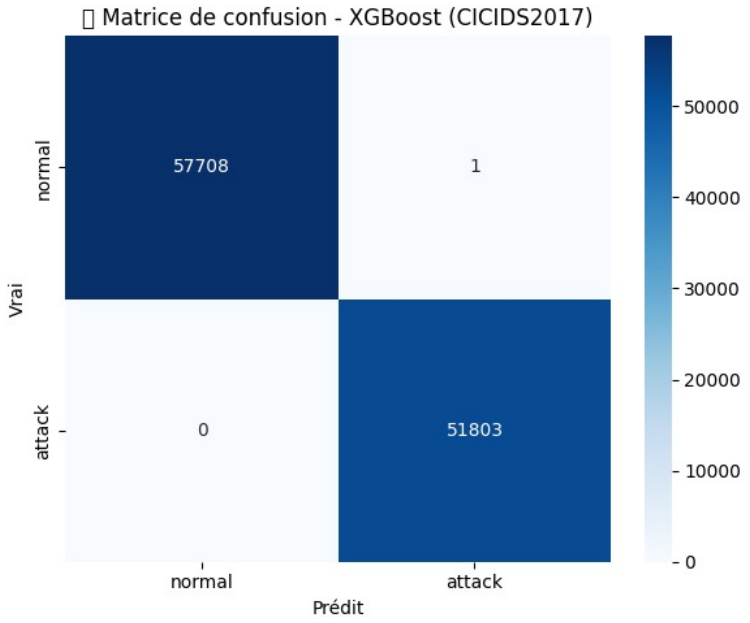


FIGURE 13 – Matrice de confusion de l'algorithme XGBoost sur CICIDS2017

De plus, la courbe ROC présente une AUC maximale de 1.00, reflétant une séparation parfaite entre les deux classes sans aucun compromis.

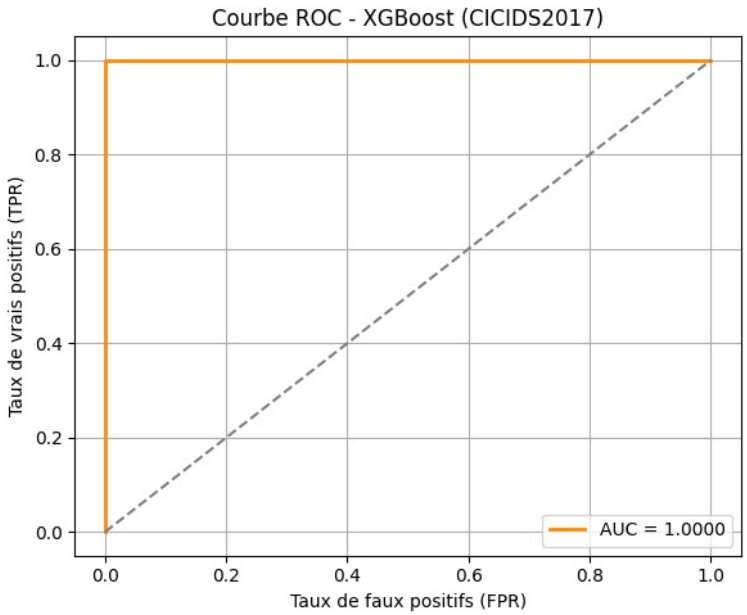


FIGURE 14 – Courbe ROC de l'algorithme XGBoost sur CICIDS2017

Ces résultats démontrent que, dans des conditions optimales, le modèle XGBoost peut atteindre un niveau de performance exceptionnel sur des données plus complexes comme CICIDS2017. Toutefois, il convient de nuancer ces résultats, car un score parfait peut être le signe d'un risque de surapprentissage ou d'un jeu de test peu diversifié. Malgré cela, cette expérimentation vient renforcer notre volonté de poursuivre une comparaison complète entre les deux datasets, dès que les ressources matérielles nécessaires seront disponibles.

Limitations matérielles et contraintes temporelles

Tout au long de notre projet, nous avons dû faire face à des contraintes techniques importantes, principalement liées à la capacité de calcul et au temps disponible. Le traitement de grands volumes de données, l'entraînement de modèles complexes et l'exploration d'approches alternatives comme le multi-classe ou l'utilisation de nouveaux jeux de données ont été fortement limités par les ressources matérielles mises à notre disposition.

Nos machines de travail, bien que suffisantes pour une première phase de développement, se sont révélées peu adaptées aux charges lourdes, ce qui a restreint notre marge de manœuvre. De plus, le cadre temporel du projet, limité par le calendrier académique, ne nous a pas permis d'approfondir certaines expérimentations qui demandaient plus de cycles d'entraînement ou de traitement.

Perspectives futures

Dans la continuité de ce projet, plusieurs pistes peuvent être explorées pour améliorer et élargir le système de détection proposé. Tout d'abord, il serait intéressant de déployer le modèle sur des infrastructures plus puissantes, afin de surmonter les limitations rencontrées. Cela permettrait notamment d'exploiter pleinement le dataset CICIDS2017, mais aussi d'appliquer des techniques avancées comme l'équilibrage des classes, l'optimisation par validation croisée extensive, ou encore l'intégration de modèles de deep learning.

En parallèle, il serait pertinent de poursuivre l'approche multi-classes pour offrir une classification plus fine et exploitable des types d'attaques, ce qui constituerait une réelle valeur ajoutée dans les environnements de cybersécurité. Une autre piste consisterait à mettre en œuvre un système de détection en temps réel, connecté directement à des flux réseau, pour des scénarios d'utilisation plus proches du terrain.

Conclusion

Cette partie met en lumière les ambitions que nous portions pour aller au-delà du périmètre initial de notre projet. L'intégration d'un dataset plus riche comme CICIDS2017 et l'approfondissement de l'analyse multi-classes sont autant de pistes qui mériteraient d'être poursuivies, une fois les barrières techniques levées.

Bien que notre travail ait été limité par le matériel et le temps disponibles, il ouvre la voie à des développements plus poussés, combinant précision, richesse des données, et application pratique dans des environnements de sécurité plus complexes et réalistes.

Bibliographie

[1] Ni Gao, Ling Gao, Quanli Gao, and Hai Wang. *An intrusion detection model based on deep belief networks*. In 2014 Second International Conference on Advanced Cloud and Big Data, pages 247–252. IEEE, 2014.

[2] A. Tesfahun, D. Lalitha Bhaskari. *Intrusion Detection using Random Forests Classifier with SMOTE and Feature Reduction*. 2013 International Conference on Cloud & Ubiquitous Computing & Emerging Technologies, IEEE.

[3] B. Brahim Embarka, A. Selyna. *Mise en place d'une solution de détection d'intrusion*. Disponible sur : `https://dspace.ummto.dz/bitstreams/b038e9b4-4bac-46fd-bfec-0c62abb53795/download`

[4] M. Houcine. *Vers une amélioration de la détection d'intrusion par les méthodes de sélection des fonctionnalités à l'aide des arbres de décision*. Mémoire de Master, 2020–2021.

[5] J. Timmis. *Artificial immune systems, A novel data analysis technique inspired by the immune network theory*. 1999.

[6] A. Pharate, H. Bhat, V. Shilimkar, and N. Mhetre. *Classification of Intrusion Detection System International Journal of Computer Applications*, May 2015.

[7] Karen Scarfone and Peter Mell. *Guide to Intrusion Detection and Prevention Systems (IDPS)*. National Institute of Standards and Technology, 2012.

[8] M. Houcine. *Vers une amélioration de la détection d'intrusion par les méthodes de sélection des fonctionnalités à l'aide des arbres de décision*. Mémoire de Master, 2020–2021.

[9] Manu Bijone, Jitendra Dangra. *International Journal for Scientific Research & Development*, 2016.

[10] `https://inventiv-it.fr/intelligence-artificielle/`, consulté le 25-05-2025.

[11] `https://www.digitalinsider.fr/article/machine-learning-definition`, consulté le 25-05-2025.

[12] `https://brightcape.co/apprentissage-supervise-vs-non-supervise/`, consulté le 26-05-2025.

[13] Datakeen, *8 Machine Learning Algorithms Explained in Human Language*, Disponible sur : `https://www.datakeen.co/8-machine-learning-algorithms-explained-in-human-language-2/`, consulté le 26 mai 2025.

[14] M. Tavallaee, E. Bagheri, W. Lu, A. A. Ghorbani, *A detailed analysis of the KDD CUP 99 data set*, Proceedings of the Second IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009.

[15] KDD Cup 1999 Data. `http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html`

[16] S. T. Salama, H. Eid, R. El-Sayed, A. Kamel, *Evaluation of the KDD99 dataset for intrusion detection systems*, International Journal of Computer, Electrical, Automation, Control and Information Engineering, Vol. 8, No. 10, 2014.

[17] https://datascientest.com/matrice-de-confusion, consulté le 26-05-2025.

[18] https://www.techno-science.net/definition/1470.html, consulté le 08-05-2025.

[19] https://www.wildcodeschool.com/, consulté le 08-05-2025.

[20] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html, consulté le 10-05-2025.

[21] https://scikit-learn.org/stable/, consulté le 10-05-2025.

[22] https://datascientest.com/en/all-about-data-pipeline, consulté le 10-05-2025.

[23] https://apmonitor.com/pds/index.php/Main/XGBoostClassifier, consulté le 10-05-2025.

[24] https://matplotlib.org/2.0.2/users/pyplot_tutorial.html, consulté le 10-05-2025.

[25] https://seaborn.pydata.org/, consulté le 10-05-2025.

[26] https://www.blackduck.com/blog/python-pickling.html, consulté le 10-05-2025.

[27] https://www.stat4decision.com/fr/creer-une-application-web-data-en-python-streamlit-vs-dash/, consulté le 12-05-2025.

[28] https://medium.com/tech-data-hub/a-deep-dive-into-the-os-library-in-python-functions-features-and-best-practices-582567bebb06, consulté le 10-05-2025.

[29] https://docs.python.org/fr/3/library/warnings.html, consulté le 10-05-2025.